

Engenharia de Software – PUC Campinas

Estrutura e Recuperação de Dados I

Prof. Dr. Paulo Cesar de Alvarenga Lucci

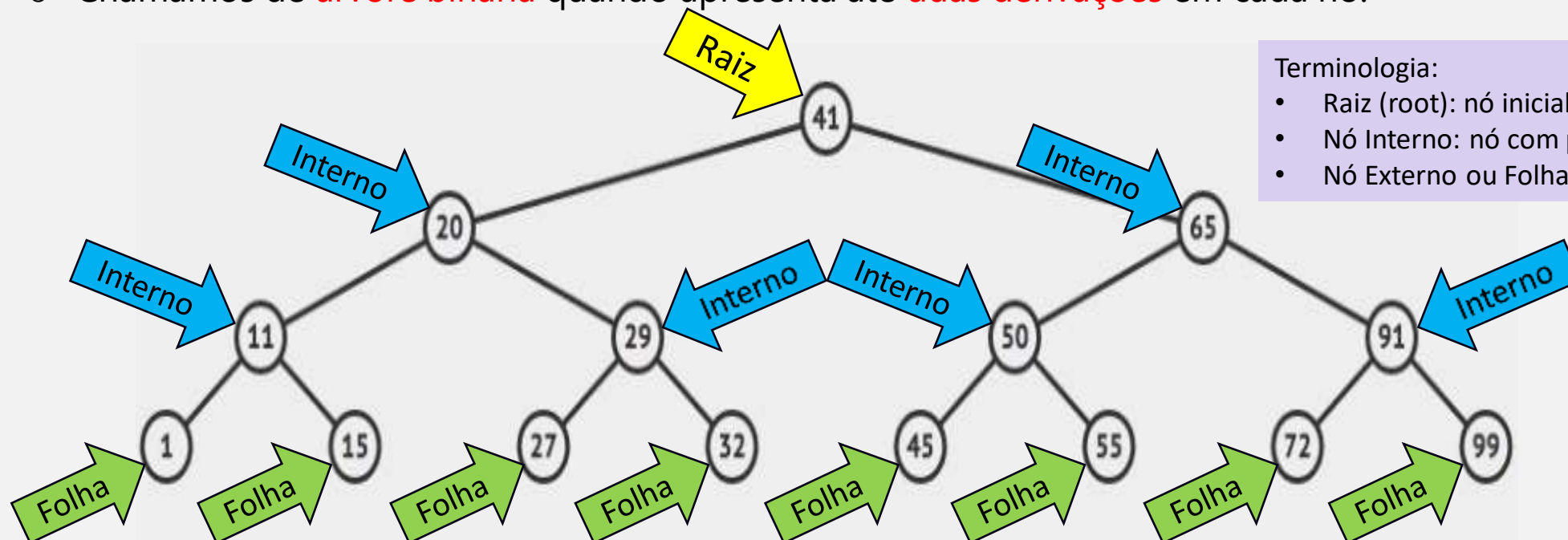
Árvores Binárias de Busca

Binary Search Tree (BST)

Árvores Binárias de Busca

○ INTRODUÇÃO

- Uma **árvore** é uma estrutura de dados não linear caracterizada por elementos denominados "**nós**", os quais guardam as informações desejadas.
- As derivações ocorrem seguindo um critério hierárquico, em que o primeiro **nó** é denominado "**raiz**" e os demais seguem por derivações de **dois** ou **mais** nós.
- Chamamos de **árvore binária** quando apresenta até **duas derivações** em cada nó.

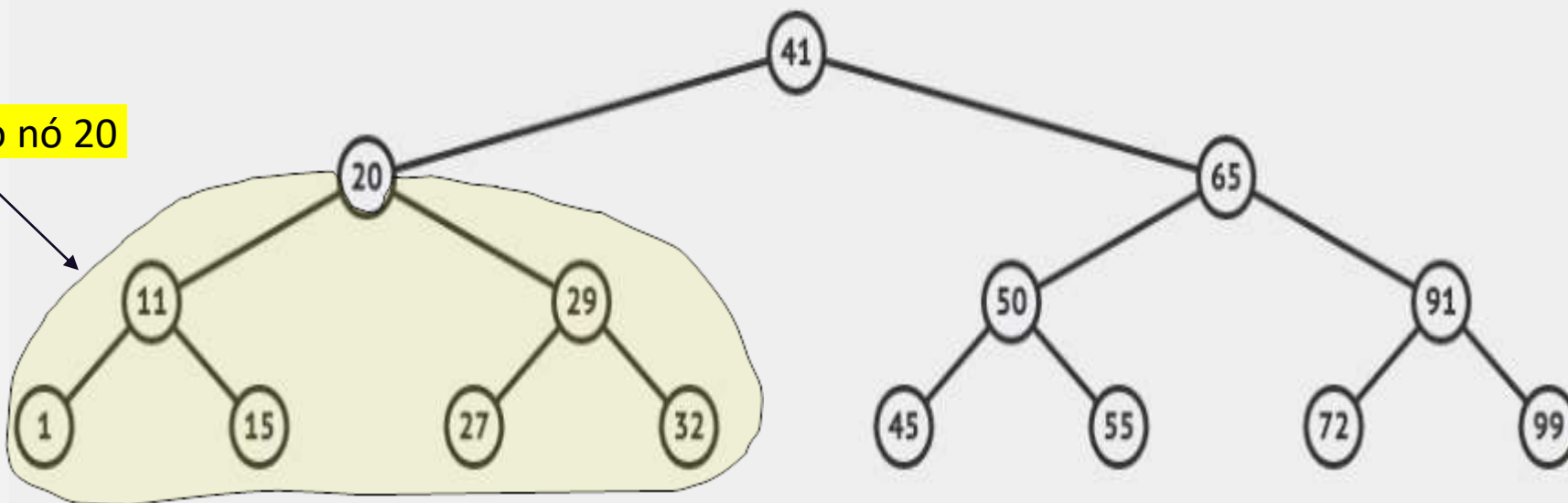


Árvores Binárias de Busca

○ PARENTESCO ENTRE NÓS

- Em árvores, utilizamos a analogia de **parentesco**:
 - Nó 20 é pai dos nós 11 e 29.
 - Nó 11 e nó 29 são nós irmãos.
 - Nó 29 e nó 50 **não** são irmãos pois **não têm o mesmo pai**.
 - Nó 15 é filho do nó 11 e irmão do nó 1.
 - Os nós 11, 1, 15, 29, 27 e 32 são descendentes do nó 20 (**subárvore do nó 20**).

Subárvore do nó 20



Árvores Binárias de Busca

- NÍVEL DO NÓ E DA ÁRVORE

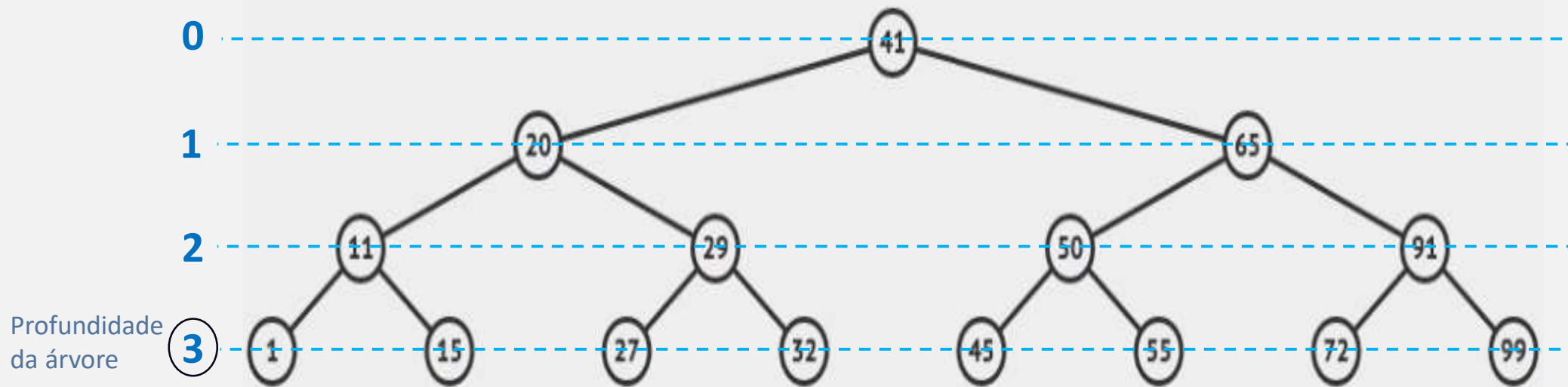
- **Nível** (ou profundidade) de um **nó**:

- O nível da raiz é 0.
 - O nível dos demais nós é igual a "nível(Pai) + 1"

- **Nível** (ou profundidade) da **árvore**:

- Maior nível dentre todos os nós.

Nível



Árvores Binárias de Busca

- GRAU DO NÓ E DA ÁRVORE

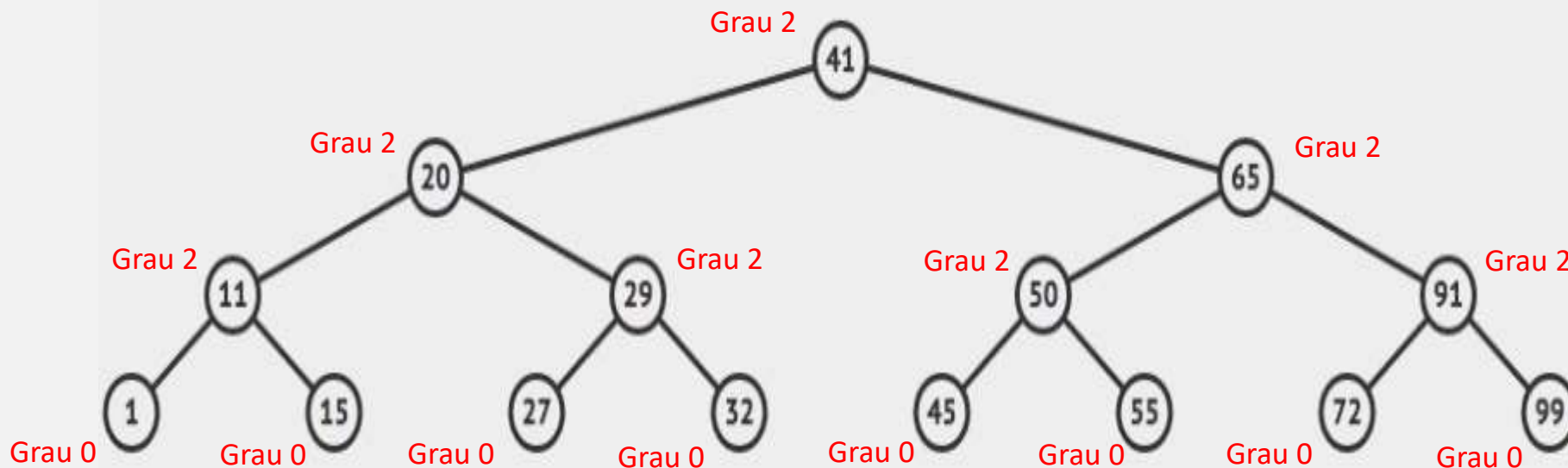
- Grau de um nó:

- Corresponde à quantidade de filhos que apresenta.

- Grau da árvore:

- É o maior entre os graus dentre todos os seus nós.

O grau dessa árvore é 2



Árvores Binárias de Busca

○ LEI DE FORMAÇÃO

- Toda árvore binária, para ser implementada, necessita de uma **Lei de Formação**, que é a regra de geração dos nós e da sua disposição na árvore, indicando quem será pai, quem será filho esquerdo, quem será filho direito.
- Uma **Lei de Formação** muito comum na inclusão de números em árvores binárias é:
 - Se $(val < Pai.val) \rightarrow$ filho à esquerda.
 - Se $(val > Pai.val) \rightarrow$ filho à direita.

Árvores Binárias de Busca

○ IMPORTANTE

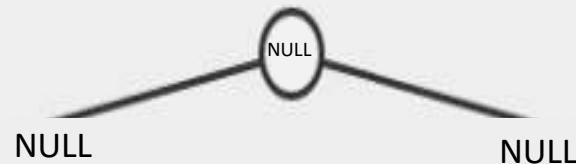
- Nos exemplos a seguir, cada **nó** da árvore armazenará, por simplicidade didática, apenas um **inteiro**.
- Na prática, porém, os **nós** costumam armazenar **estruturas de dados** mais **complexas**.
 - A **Lei de Formação** utilizará, portanto, algum **atributo** dessa estrutura para estabelecer a hierarquia.
 - Esse parâmetro chamamos de **chave** (**key**).
- Um exemplo prático é a implementação de uma agenda de contatos (nome, celular, aniversário, endereço etc).
 - O **nome do contato** define a hierarquia (exemplos: Ana < Beatriz, João > Felipe etc).
 - **Atenção:** isso **não significa** que os contatos estarão organizados em **ordem alfabética na árvore**.
 - A estrutura da árvore visa **otimizar operações** como inserção, remoção e busca de dados, e não a **ordenação visual dos elementos**.

Árvores Binárias de Busca

○ INSERÇÃO

- Suponha que vou inserir os seguintes números: { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }
- Se o nó estiver vazio → insere (e acabou).
- Caso contrário:
 - Se $(val < Pai.val)$ → filho à esquerda.
 - Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO



- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

- Se o nó estiver vazio → insere (e acabou).
- Caso contrário:
 - Se $(val < Pai.val)$ → filho à esquerda.
 - Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO

○ { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

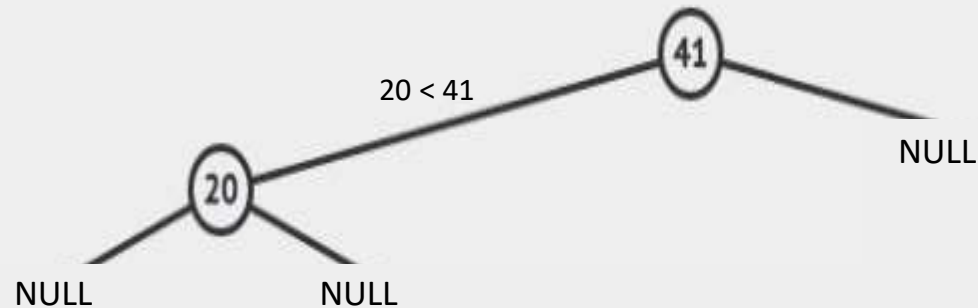
○ Se o nó estiver vazio → insere (e acabou).

○ Caso contrário:

○ Se $(val < Pai.val)$ → filho à esquerda.

○ Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



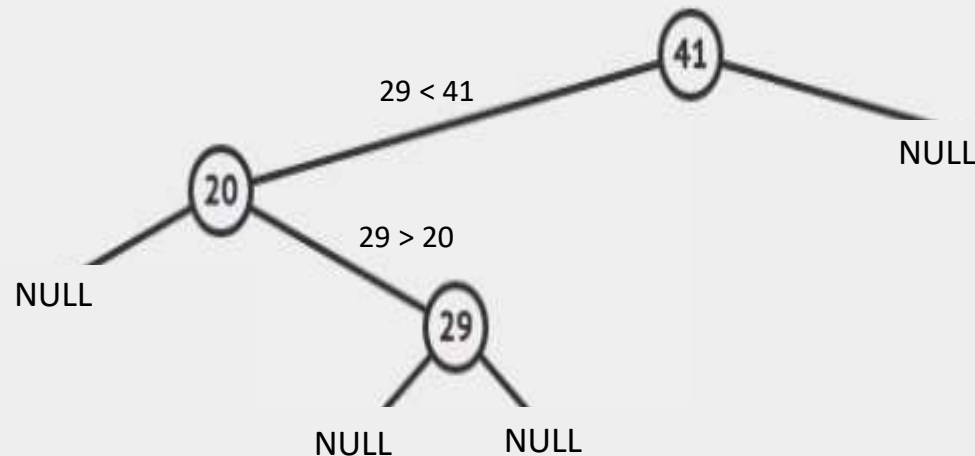
Árvores Binárias de Busca

○ INSERÇÃO

- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

- Se o nó estiver vazio → insere (e acabou).
- Caso contrário:
 - Se $(val < Pai.val)$ → filho à esquerda.
 - Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



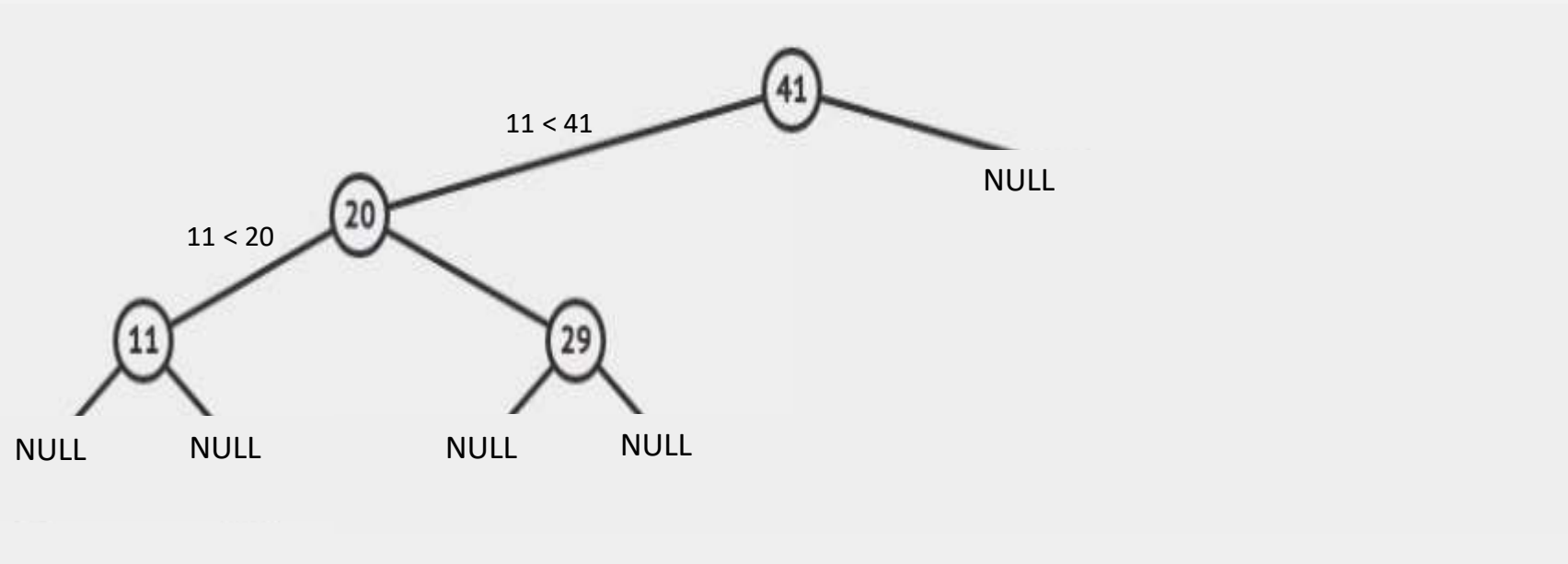
Árvores Binárias de Busca

○ INSERÇÃO

- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

- Se o nó estiver vazio → insere (e acabou).
- Caso contrário:
 - Se $(val < Pai.val)$ → filho à esquerda.
 - Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO

- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

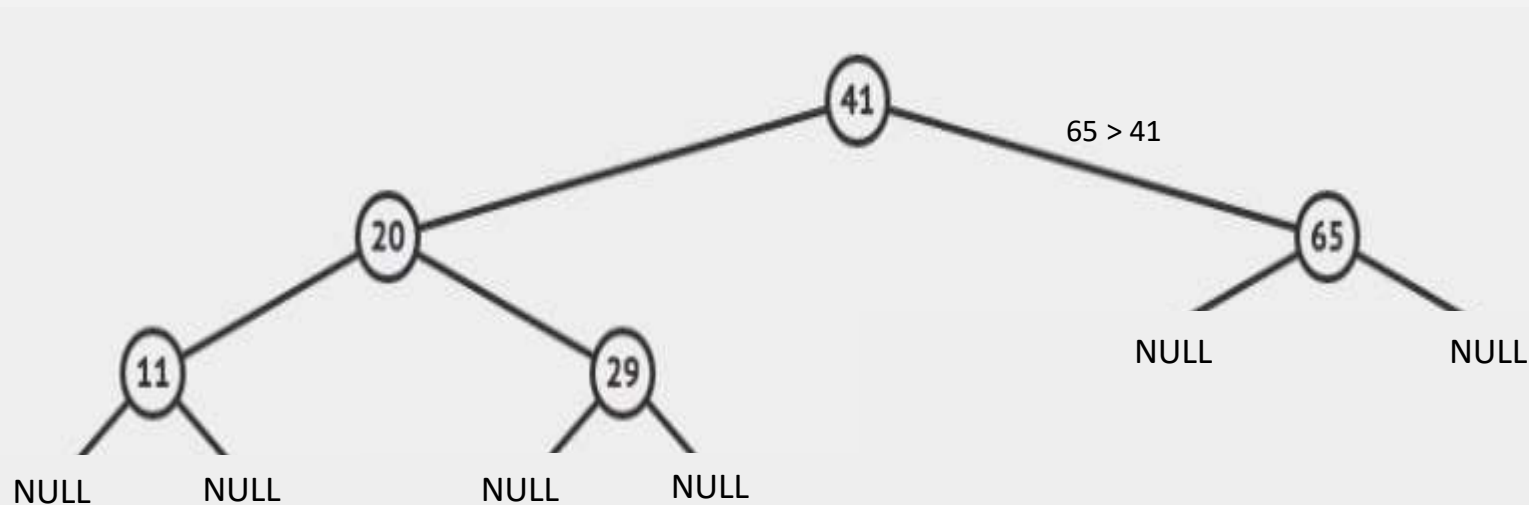
- Se o nó estiver vazio → insere (e acabou).

- Caso contrário:

- Se $(val < Pai.val)$ → filho à esquerda.

- Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



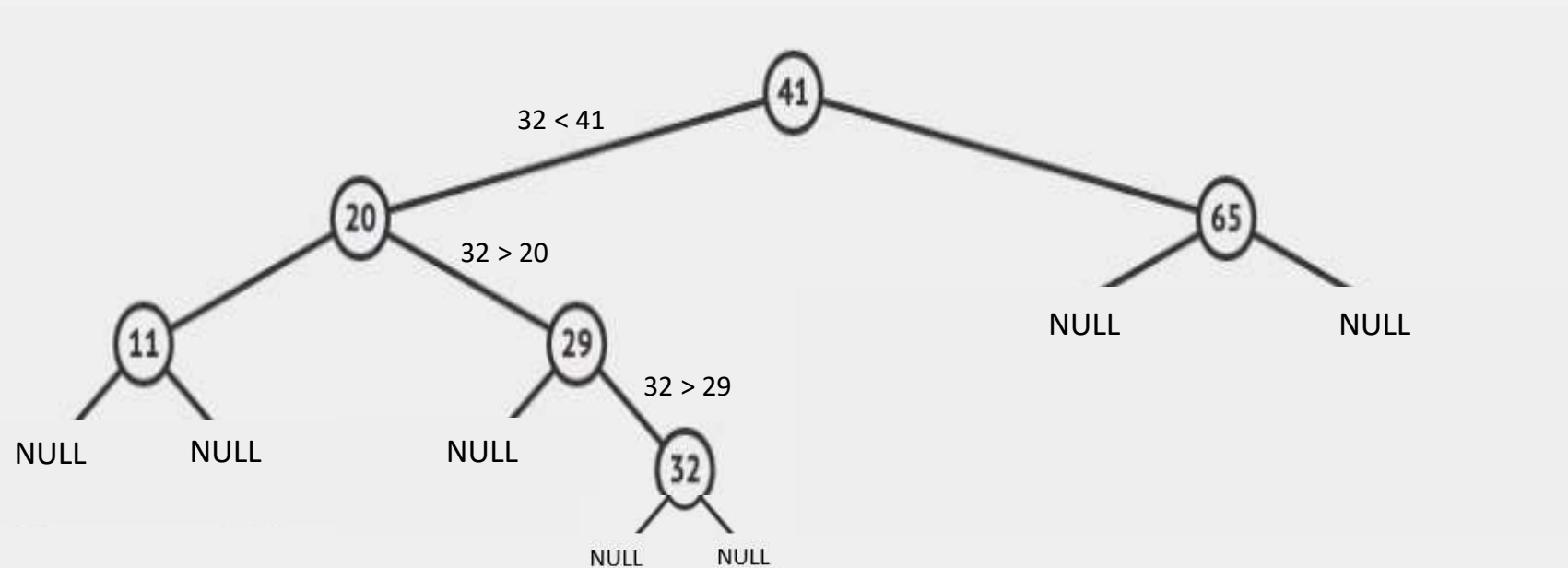
Árvores Binárias de Busca

○ INSERÇÃO

- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

- Se o nó estiver vazio → insere (e acabou).
- Caso contrário:
 - Se $(val < Pai.val)$ → filho à esquerda.
 - Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



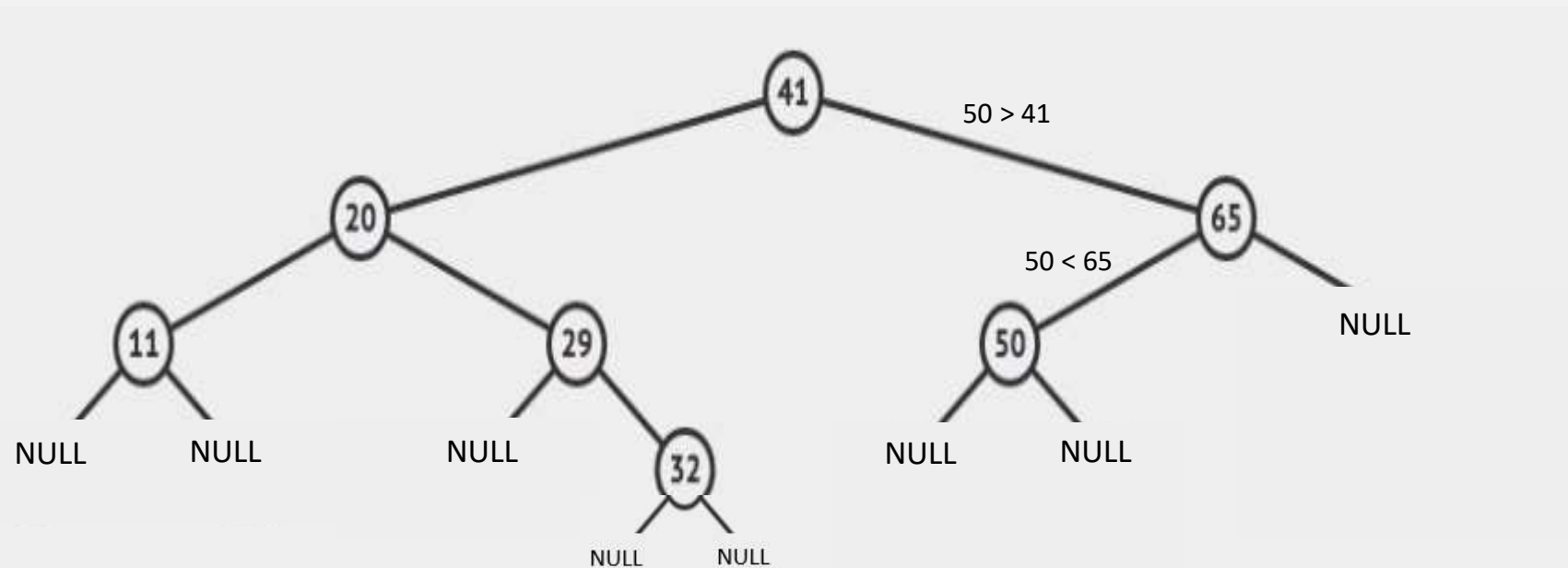
Árvores Binárias de Busca

○ INSERÇÃO

- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

- Se o nó estiver vazio → insere (e acabou).
- Caso contrário:
 - Se $(val < Pai.val)$ → filho à esquerda.
 - Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



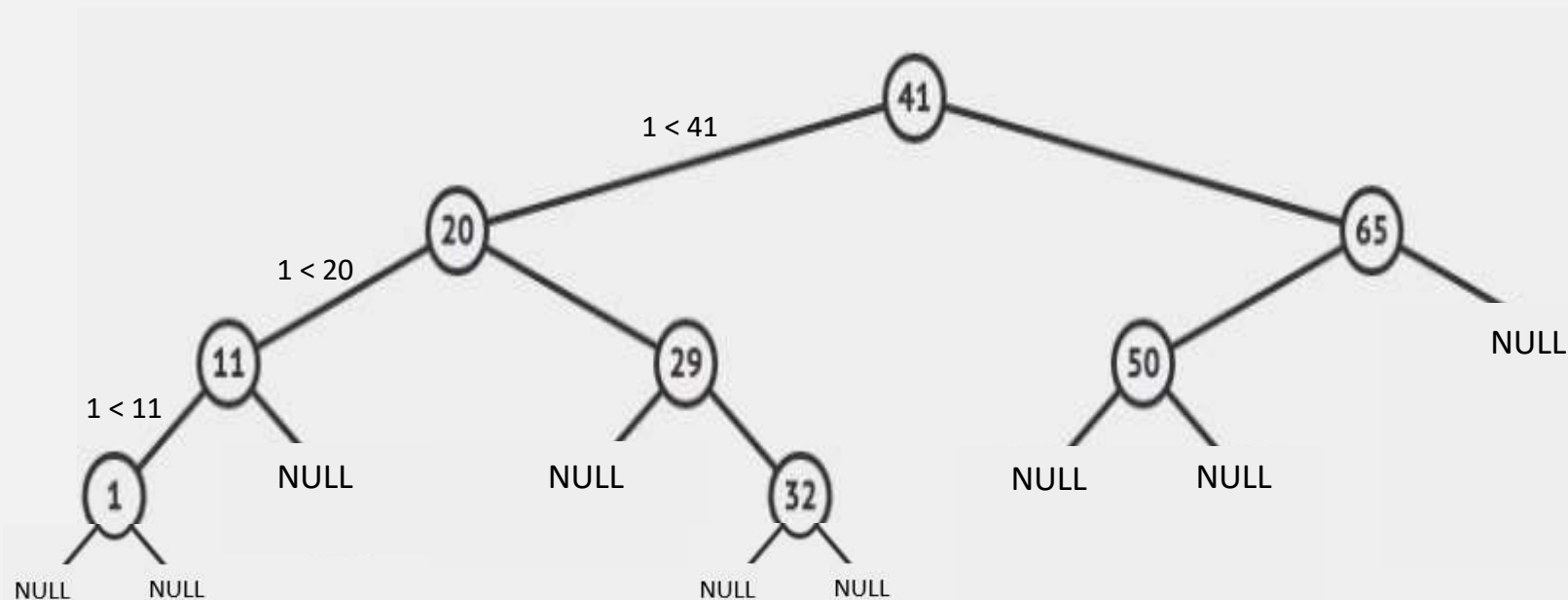
Árvores Binárias de Busca

○ INSERÇÃO

- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

- Se o nó estiver vazio → insere (e acabou).
- Caso contrário:
 - Se $(val < Pai.val)$ → filho à esquerda.
 - Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO

○ { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

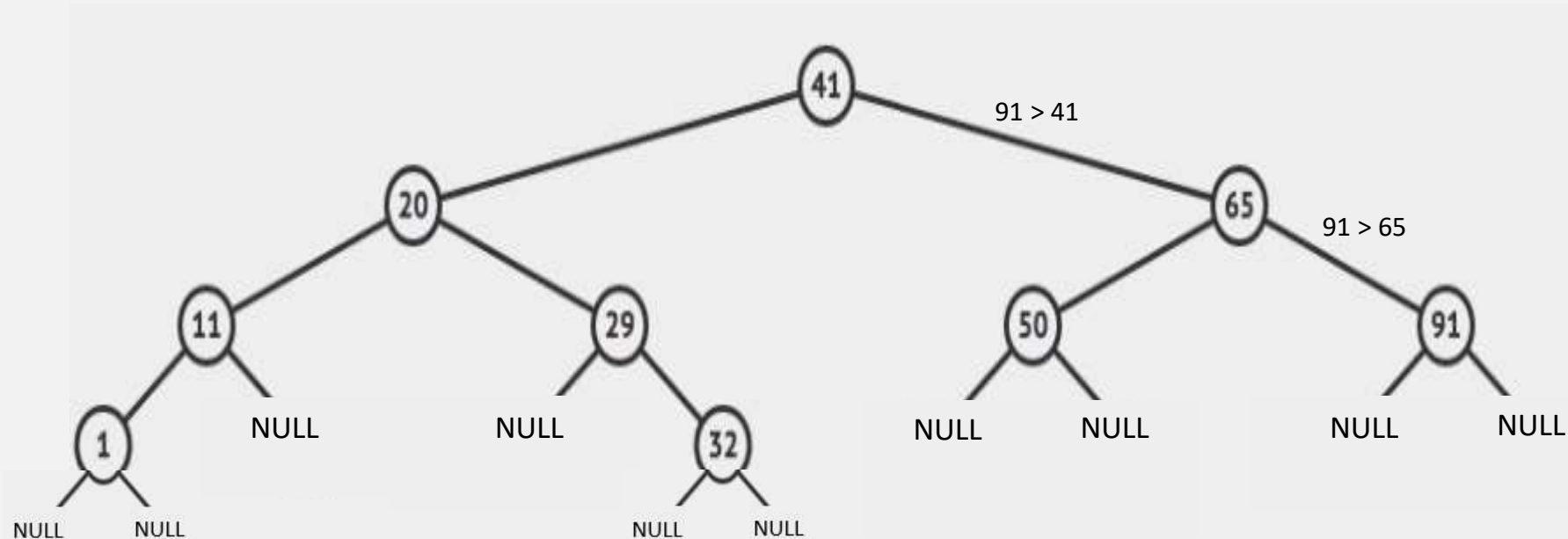
○ Se o nó estiver vazio → insere (e acabou).

○ Caso contrário:

○ Se $(val < Pai.val)$ → filho à esquerda.

○ Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO

○ { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

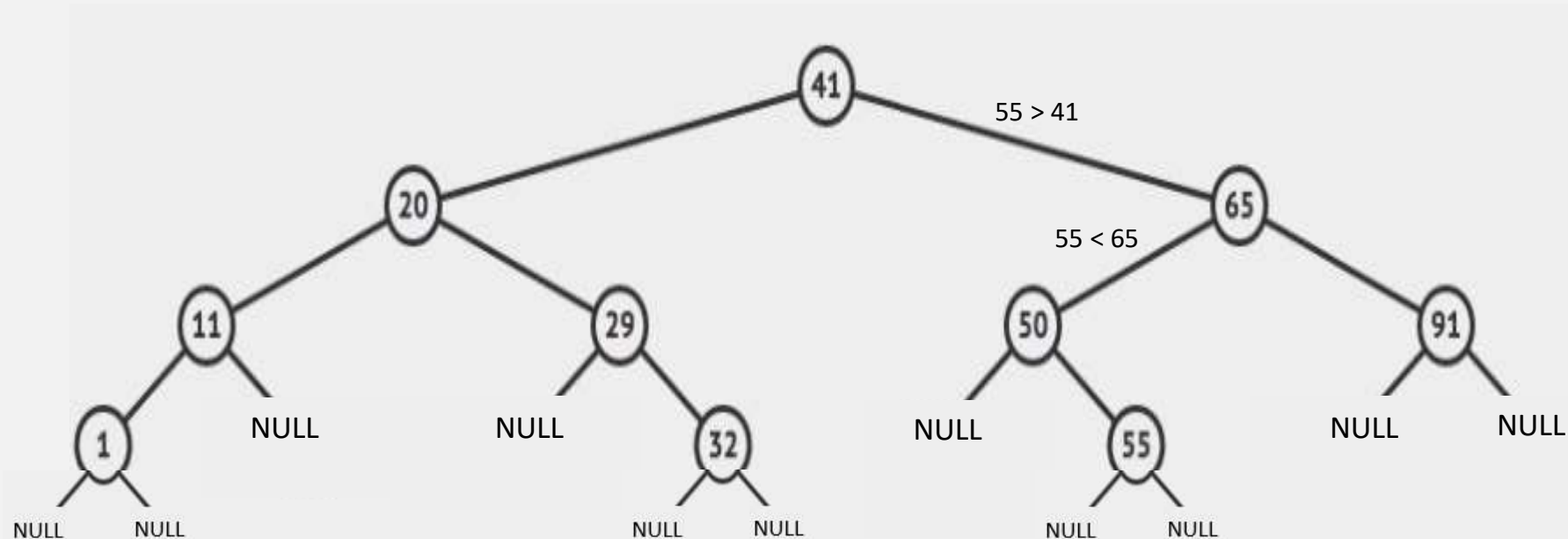
○ Se o nó estiver vazio → insere (e acabou).

○ Caso contrário:

○ Se $(val < Pai.val)$ → filho à esquerda.

○ Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

- **INSERÇÃO**

- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

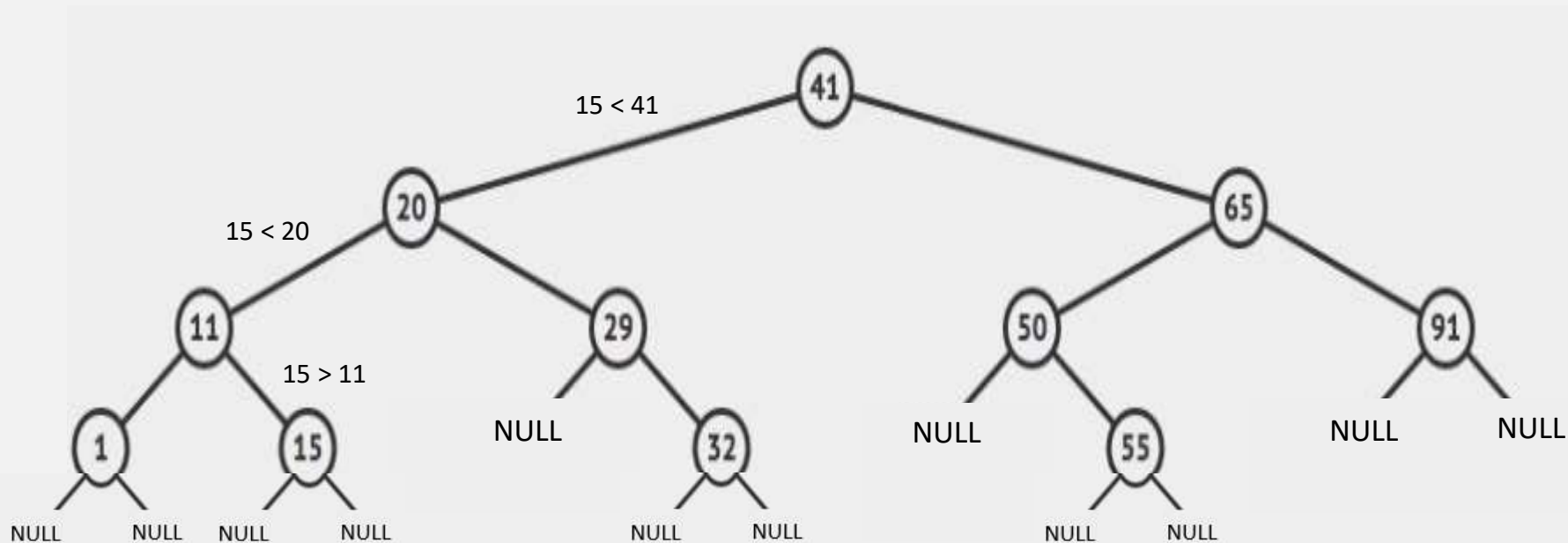
- Se o nó estiver vazio → insere (e acabou).

- Caso contrário:

- Se ($val < Pai.val$) $\rightarrow$ filho à esquerda.

- Se ($val > Pai.val$) $\rightarrow$ filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO

- { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

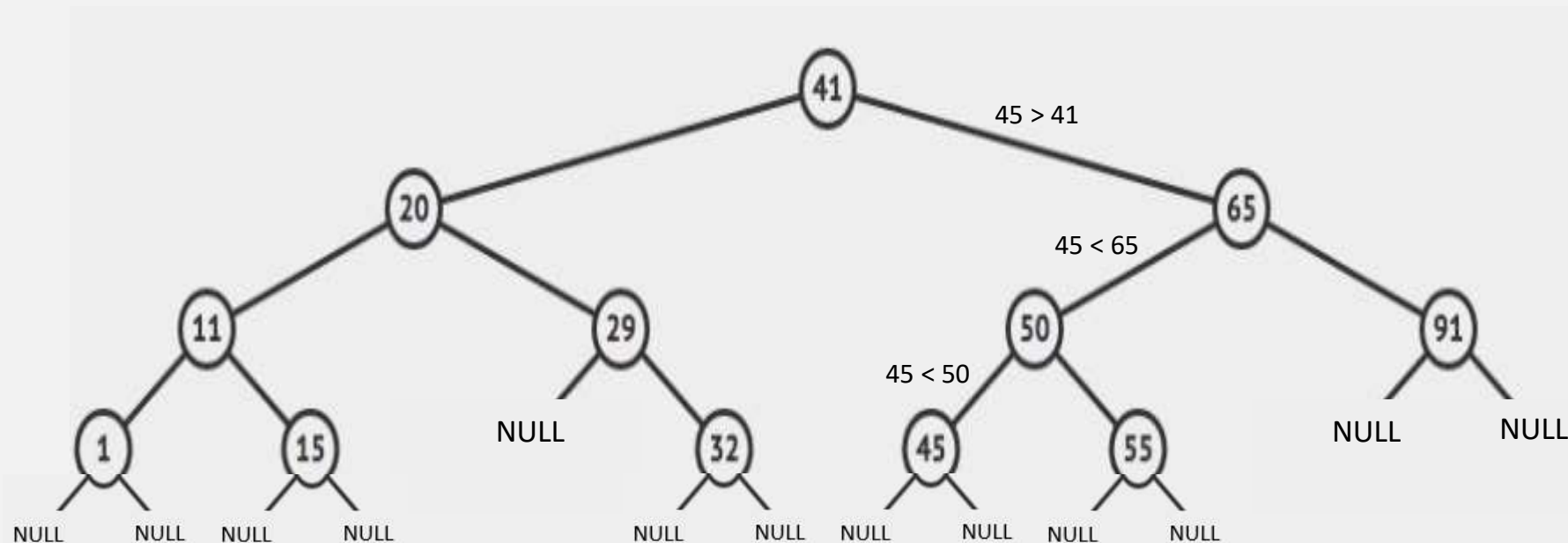
- Se o nó estiver vazio → insere (e acabou).

- Caso contrário:

- Se $(val < Pai.val)$ → filho à esquerda.

- Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO

○ { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

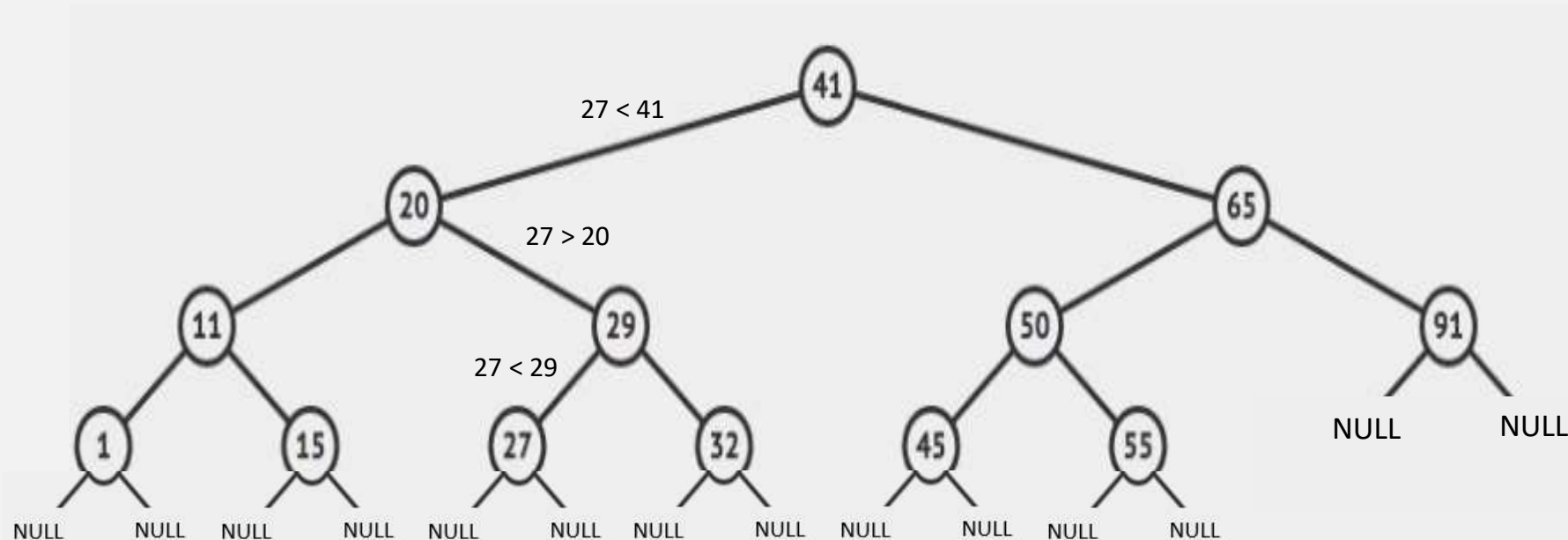
○ Se o nó estiver vazio → insere (e acabou).

○ Caso contrário:

○ Se $(val < Pai.val)$ → filho à esquerda.

○ Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO

○ { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 }

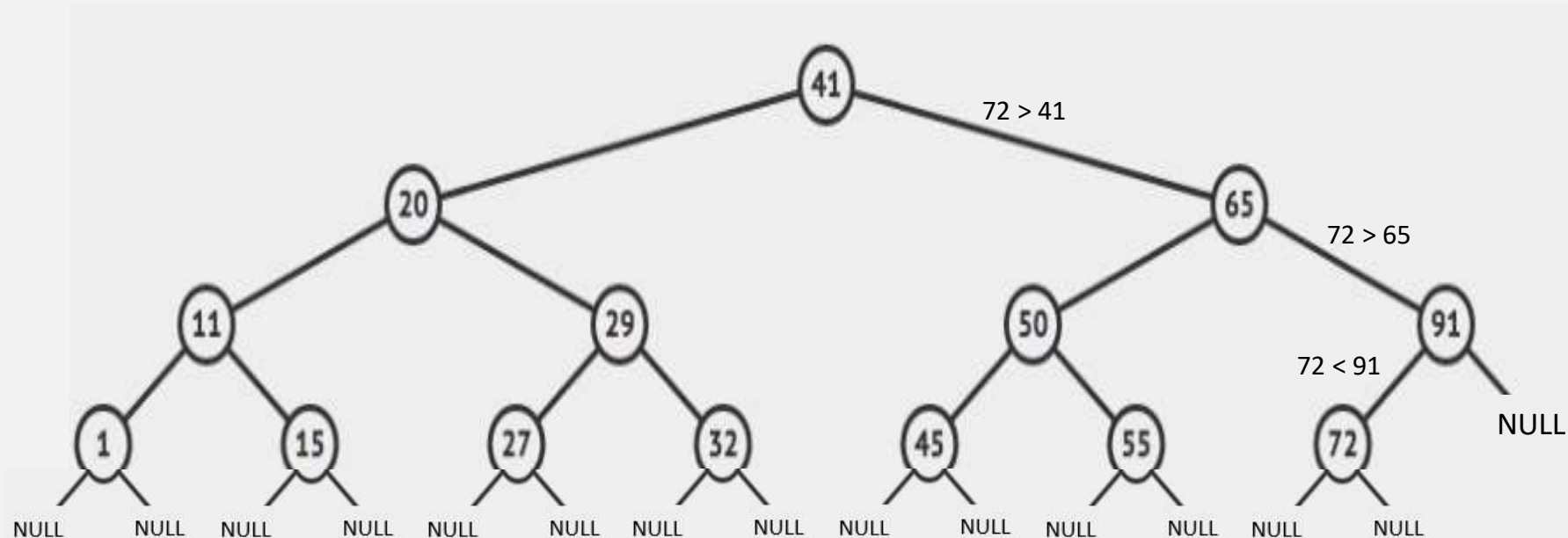
○ Se o nó estiver vazio → insere (e acabou).

○ Caso contrário:

○ Se $(val < Pai.val)$ → filho à esquerda.

○ Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.



Árvores Binárias de Busca

○ INSERÇÃO

○ { 41, 20, 29, 11, 65, 32, 50, 1, 91, 55, 15, 45, 27, 72, 99 } 

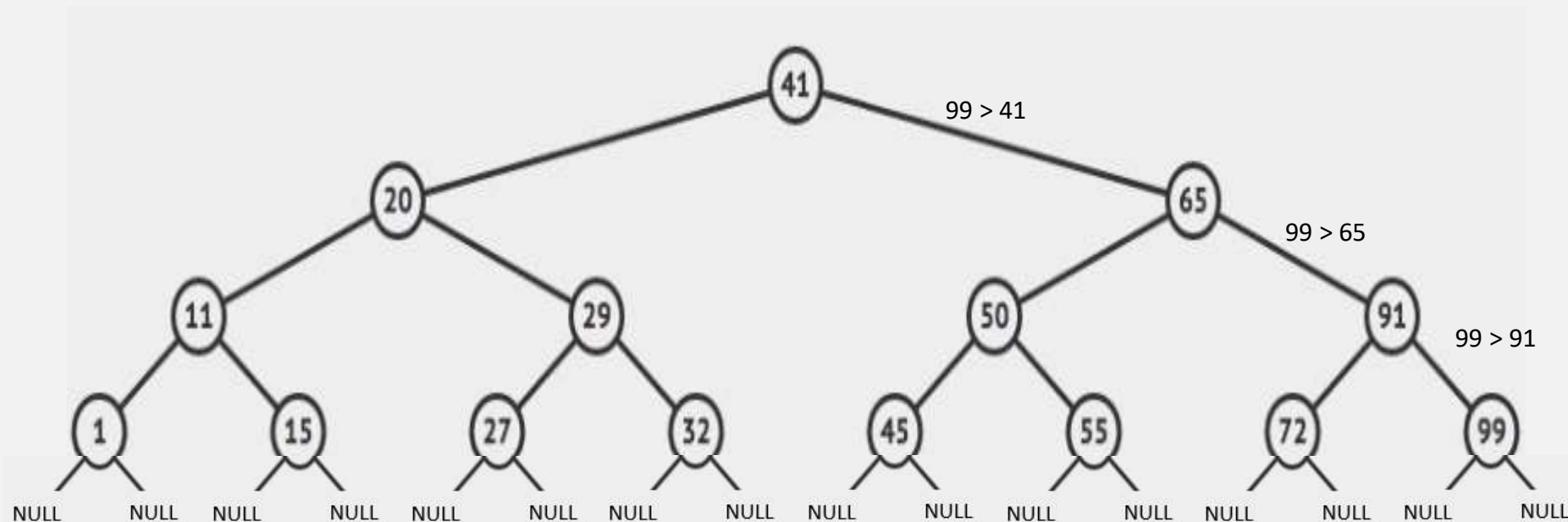
○ Se o nó estiver vazio → insere (e acabou).

○ Caso contrário:

○ Se $(val < Pai.val)$ → filho à esquerda.

○ Se $(val > Pai.val)$ → filho à direita.

Entendam o algoritmo como a busca pela primeira posição disponível (NULL), obedecendo a Lei de Formação.

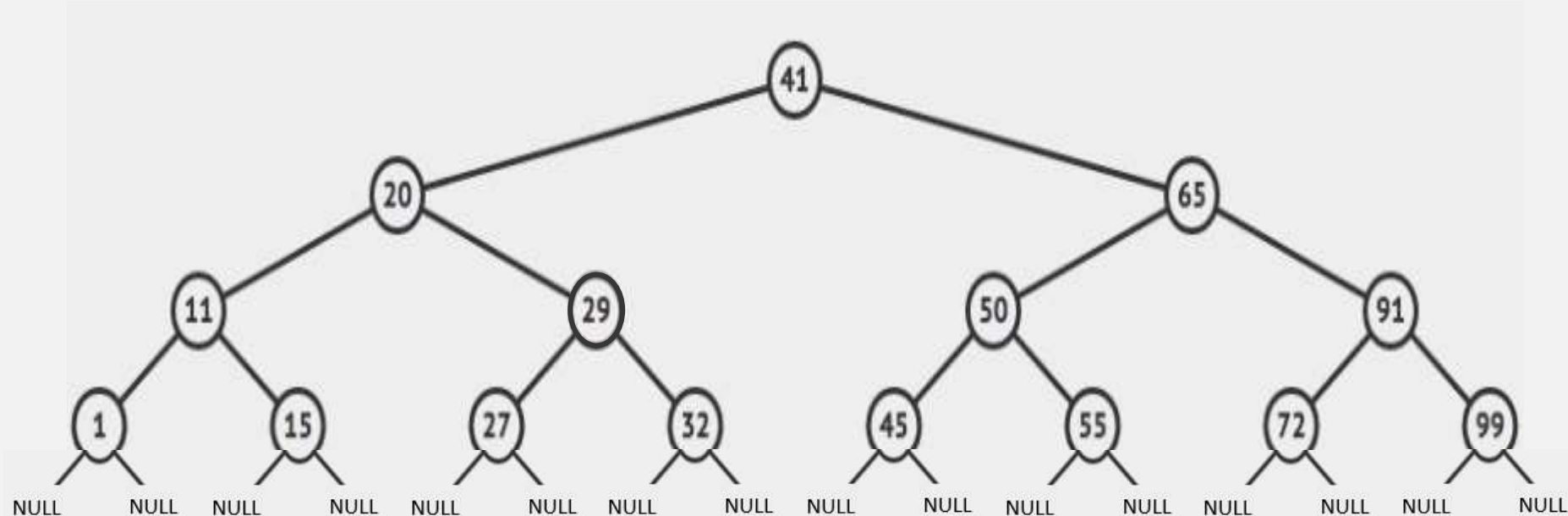


Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
 - Se o nó estiver nulo → key não encontrada!
 - Se o nó tiver a key buscada → key encontrada!
 - Caso contrário:
 - Se $(val < Pai.val)$ → filho_à esquerda.
 - Se $(val > Pai.val)$ → filho_à direita.

Exemplo: Busca pelo número 27:



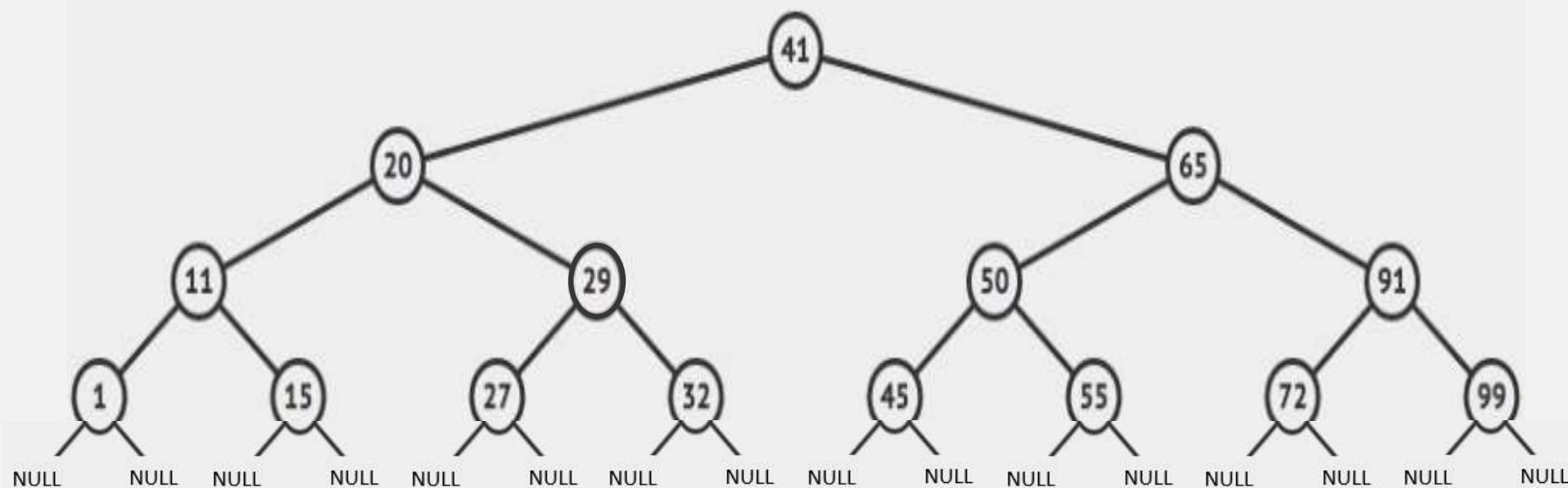
Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
 - Se o nó estiver nulo → key não encontrada!
 - Se o nó tiver a key buscada → key encontrada!
 - Caso contrário:
 - Se $(val < Pai.val)$ → filho_à esquerda.
 - Se $(val > Pai.val)$ → filho_à direita.

Exemplo: Busca pelo número 27:

Achei? Não!

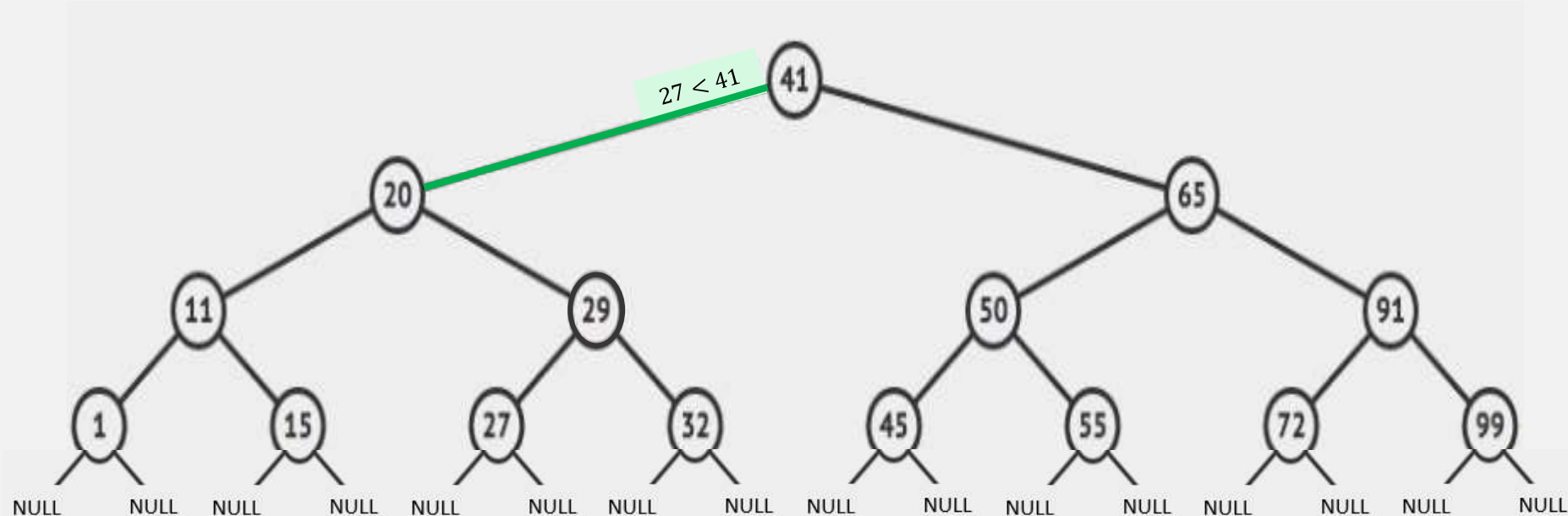


Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
 - Se o nó estiver nulo → key não encontrada!
 - Se o nó tiver a key buscada → key encontrada!
 - Caso contrário:
 - Se $(val < Pai.val)$ → filho_à esquerda.
 - Se $(val > Pai.val)$ → filho_à direita.

Exemplo: Busca pelo número 27:

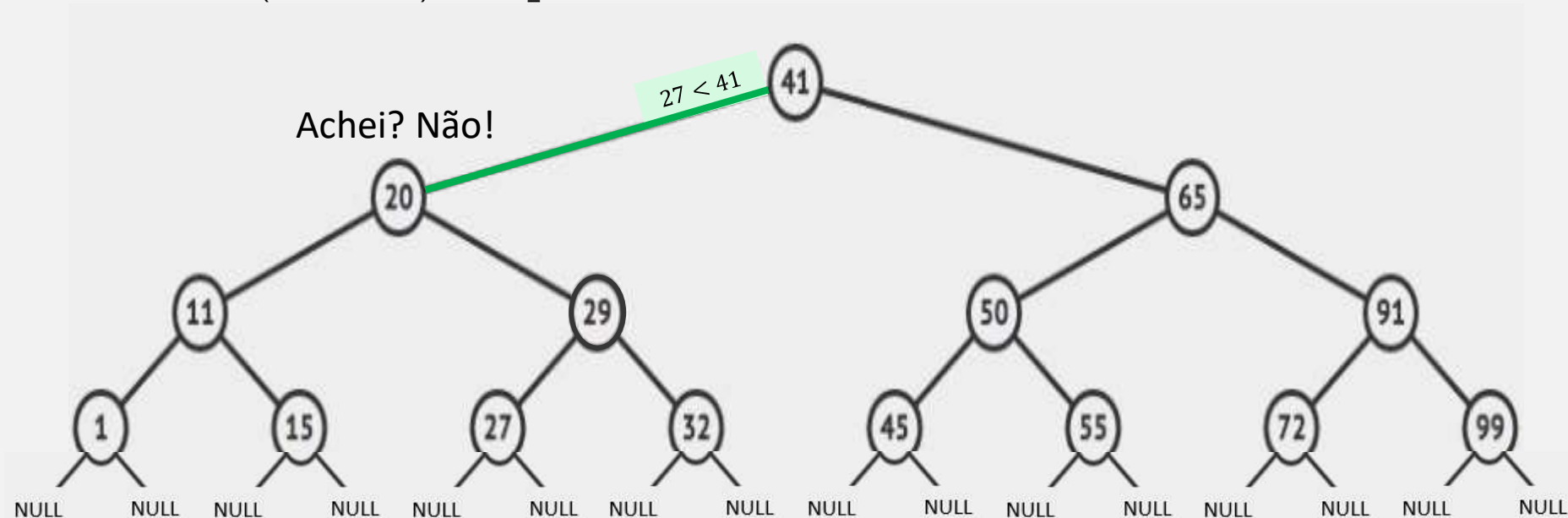


Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
 - Se o nó estiver nulo → key não encontrada!
 - Se o nó tiver a key buscada → key encontrada!
 - Caso contrário:
 - Se $(val < Pai.val)$ → filho_à esquerda.
 - Se $(val > Pai.val)$ → filho_à direita.

Exemplo: Busca pelo número 27:

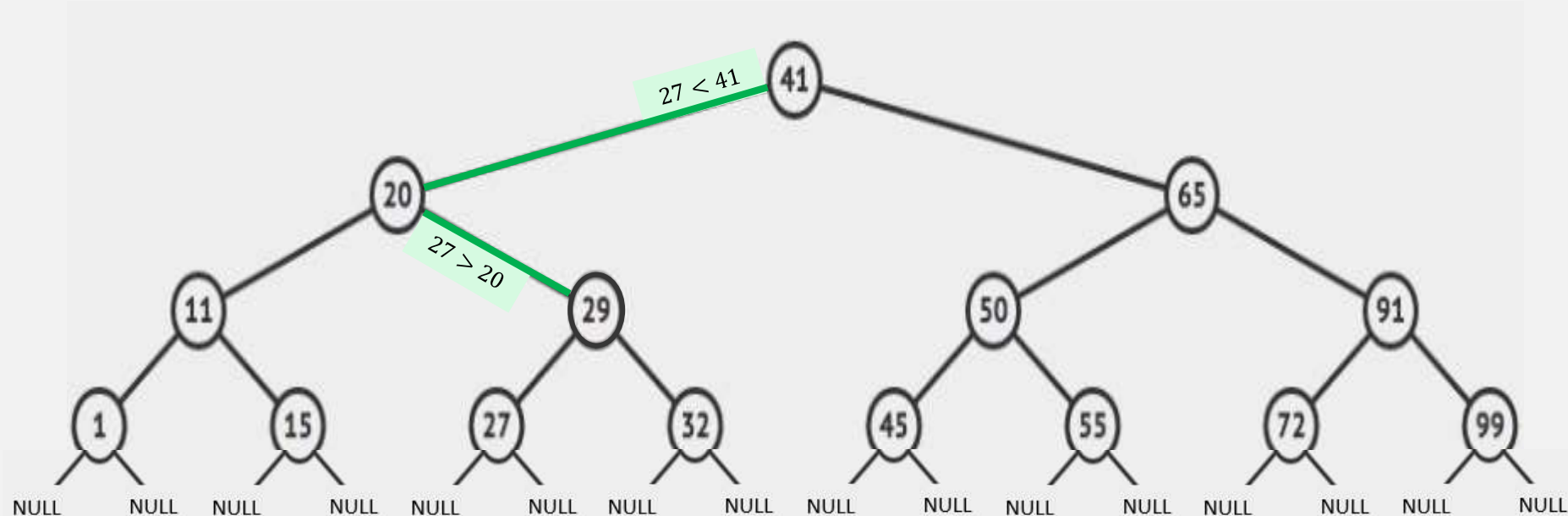


Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
 - Se o nó estiver nulo → key não encontrada!
 - Se o nó tiver a key buscada → key encontrada!
 - Caso contrário:
 - Se $(val < Pai.val)$ → filho_à esquerda.
 - Se $(val > Pai.val)$ → filho_à direita.

Exemplo: Busca pelo número 27:

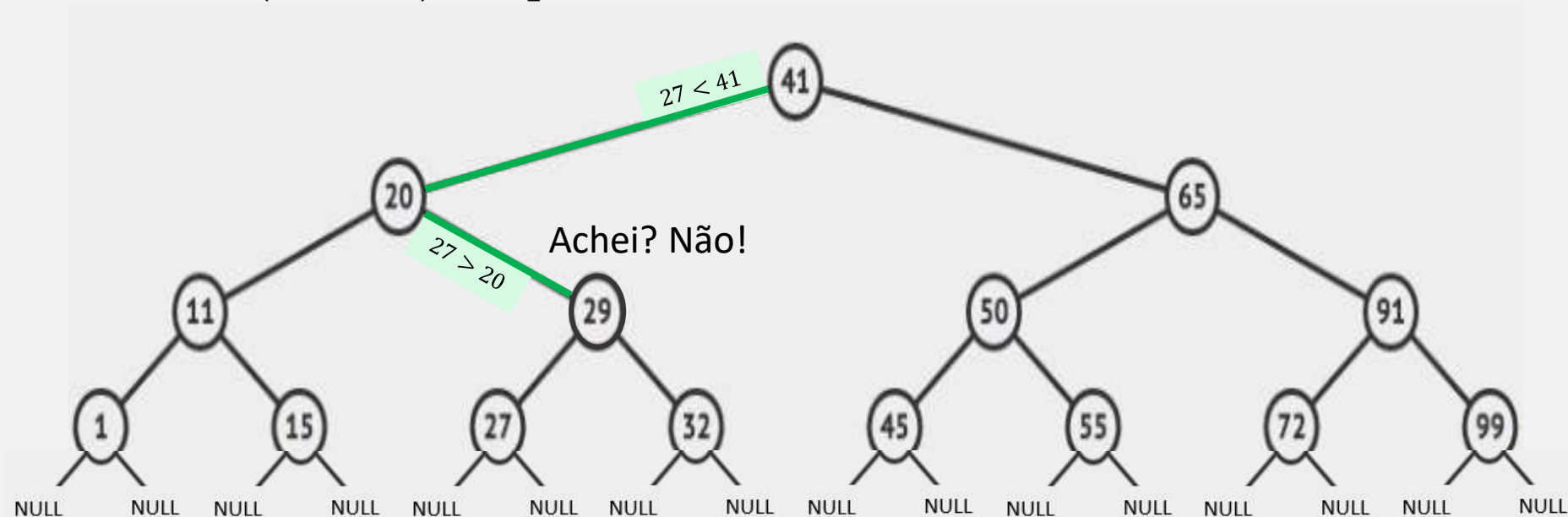


Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
 - Se o nó estiver nulo → key não encontrada!
 - Se o nó tiver a key buscada → key encontrada!
 - Caso contrário:
 - Se $(val < Pai.val)$ → filho_à esquerda.
 - Se $(val > Pai.val)$ → filho_à direita.

Exemplo: Busca pelo número 27:

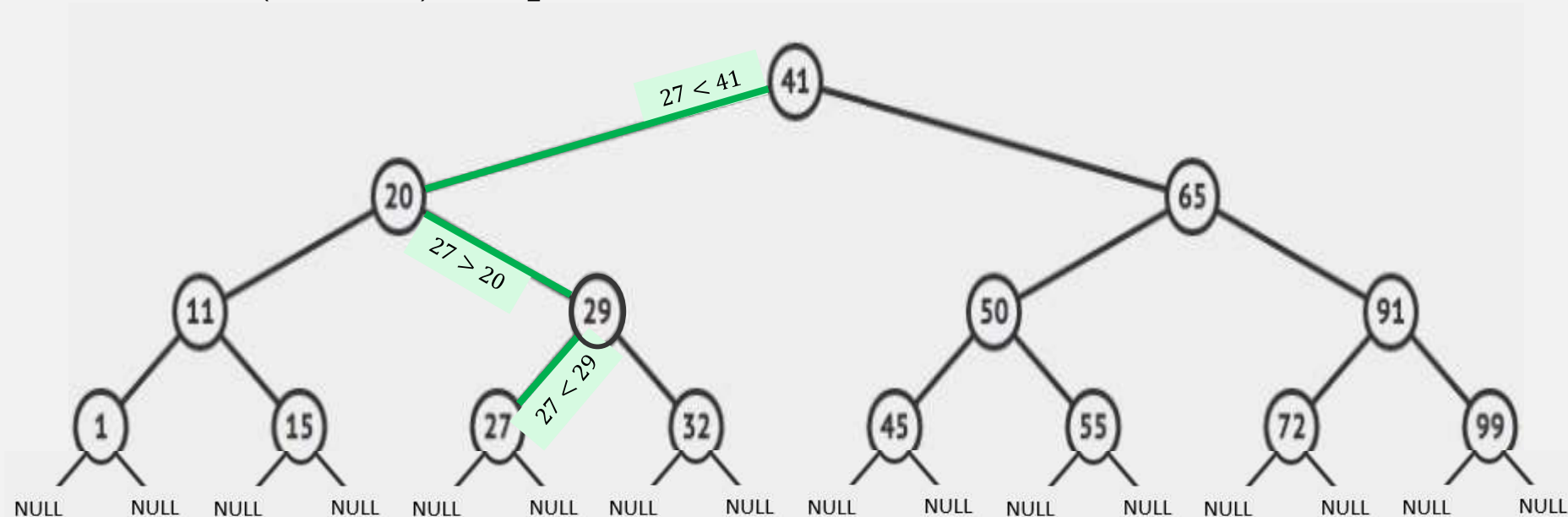


Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
 - Se o nó estiver nulo → key não encontrada!
 - Se o nó tiver a key buscada → key encontrada!
 - Caso contrário:
 - Se $(val < Pai.val)$ → filho_à esquerda.
 - Se $(val > Pai.val)$ → filho_à direita.

Exemplo: Busca pelo número 27:

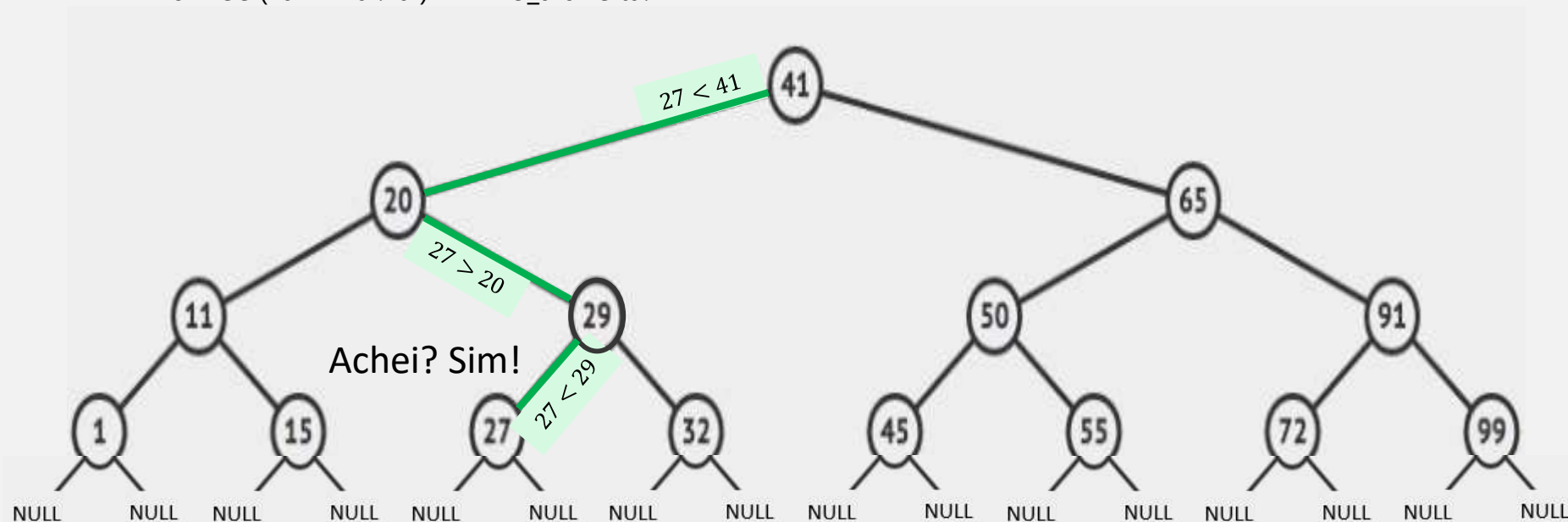


Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
 - Se o nó estiver nulo → key não encontrada!
 - Se o nó tiver a key buscada → key encontrada!
 - Caso contrário:
 - Se $(val < Pai.val)$ → filho_à esquerda.
 - Se $(val > Pai.val)$ → filho_à direita.

Exemplo: Busca pelo número 27:

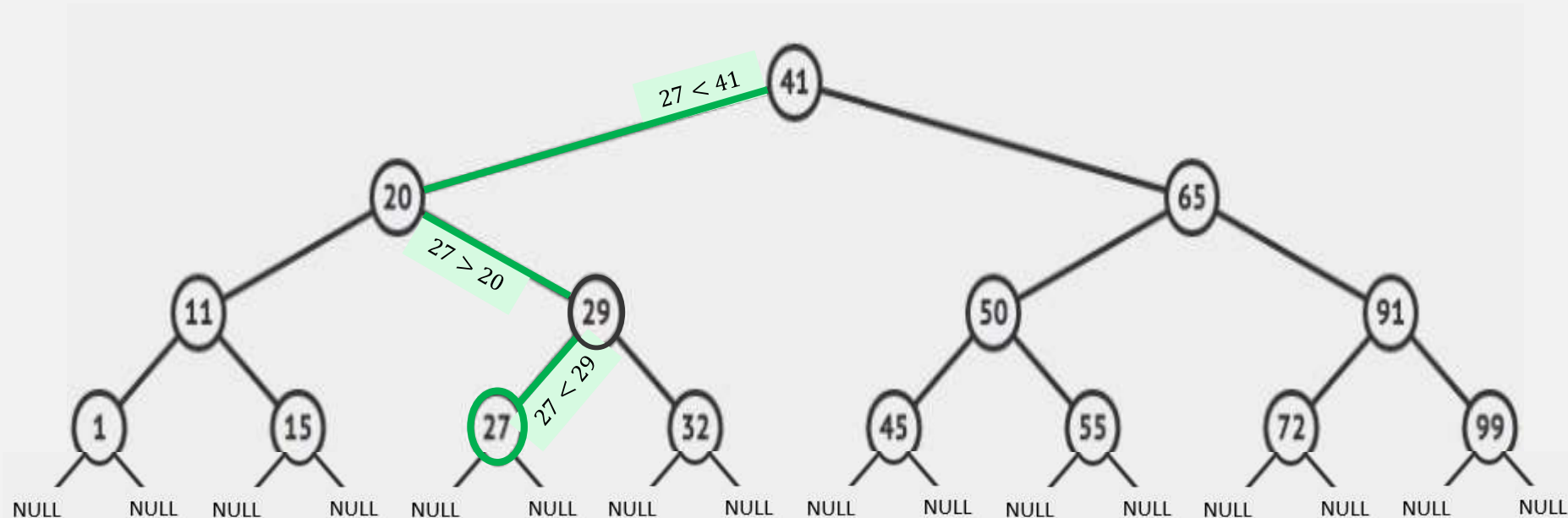


Árvores Binárias de Busca

○ BUSCA

- A mesma Lei de Formação se aplica na busca de elementos na árvore, sempre começando pela raiz.
- Se ainda não encontrou:
 - Se $(val < Pai.val) \rightarrow$ filho_à esquerda.
 - Se $(val > Pai.val) \rightarrow$ filho_à direita.

Exemplo: Busca pelo número 27:



Árvores Binárias de Busca

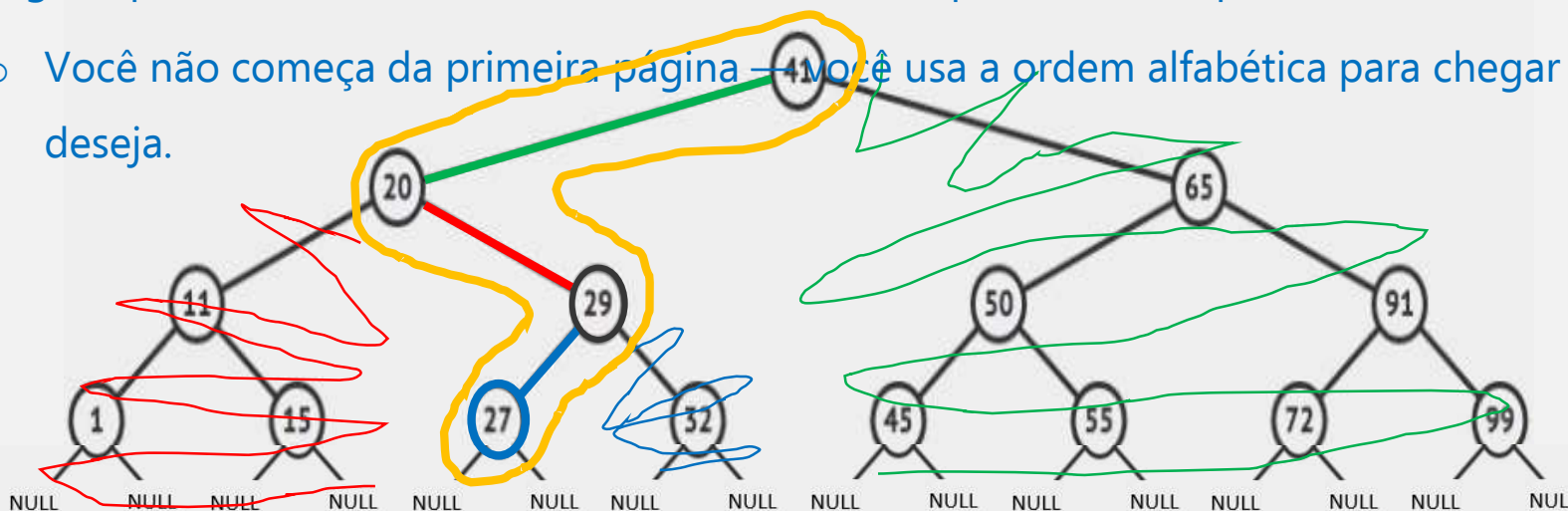
○ EFICIÊNCIA DA BUSCA E INSERÇÃO UMA ÁRVORE BINÁRIA

- Tanto na inserção quanto na busca, não houve a necessidade de percorrer todos os elementos, como faríamos em um vetor. A cada descida à esquerda ou à direita, **descartamos metade das possibilidades de cada subárvore**.

- Por exemplo: na busca, em **apenas 4 comparações** (41, 20, 29, 27), encontramos o número — isso em um conjunto com 15 elementos!

- Analogia rápida: Buscar em uma árvore binária é como procurar uma palavra em um **dicionário físico**:

- Você não começa da primeira página — **Mostramos a ordem alfabética para chegar mais rápido onde deseja.**



Árvores Binárias de Busca

○ REMOÇÃO

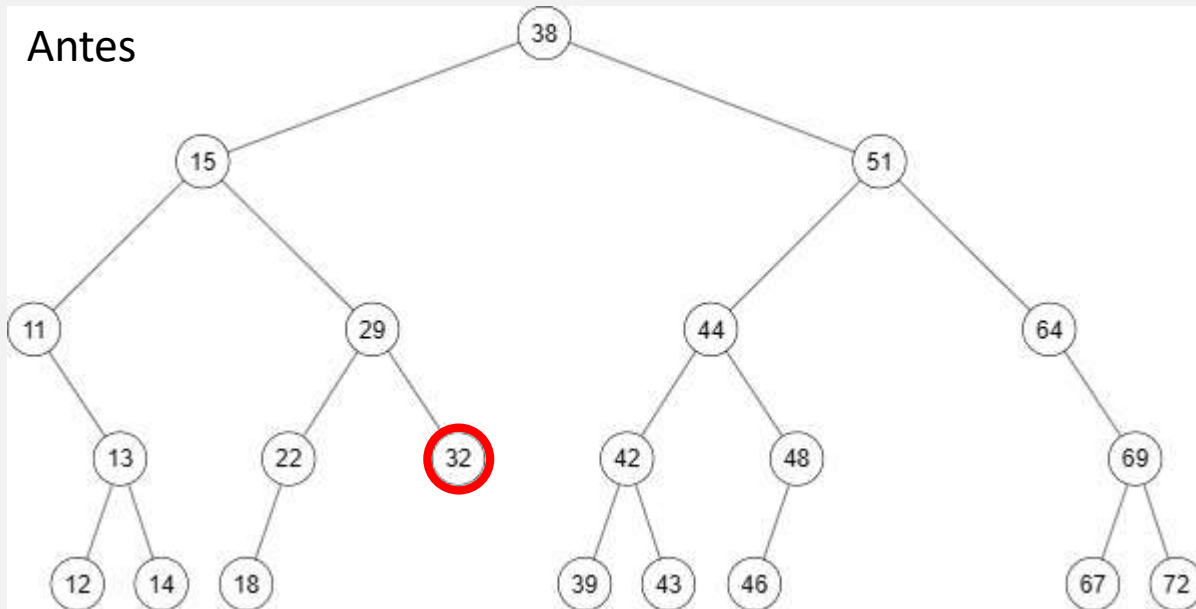
- A remoção de um nó de uma árvore binária implica em **reorganizar** os nós da subárvore que sobra para que não fiquem nós de conexão vazios (também fizemos assim na remoção de uma posição intermediária de um vetor).
- Existem 3 situações possíveis:
 1. **Remoção de nó folha:** simplesmente removemos o nó (deixa de existir sem remanejamento).
 2. **Remoção de nó com apenas 1 filho:** o filho (e seus descendentes) sobem 1 nível, ocupando o lugar de seu pai (que será removido).
 3. **Remoção de nós com 2 filhos:** Precisamos encontrar um substituto para o nó removido.
 - Estratégia:
 - O nó removido será substituído pelo **sucessor** (menor valor da subárvore direita) ou pelo **antecessor** (maior valor da subárvore esquerda).
- PELO FATO DE CADA REMOÇÃO DE NÓS COM 2 FILHOS IMPLICAR EM **MOVER** UM NÓ EXISTENTE PARA OCUPAR O LUGAR DO NÓ REMOVIDO, ESSA MOVIMENTAÇÃO IMPLICA EM UMA NOVA REMOÇÃO (DO NÓ MOVIDO) →

Árvores Binárias de Busca

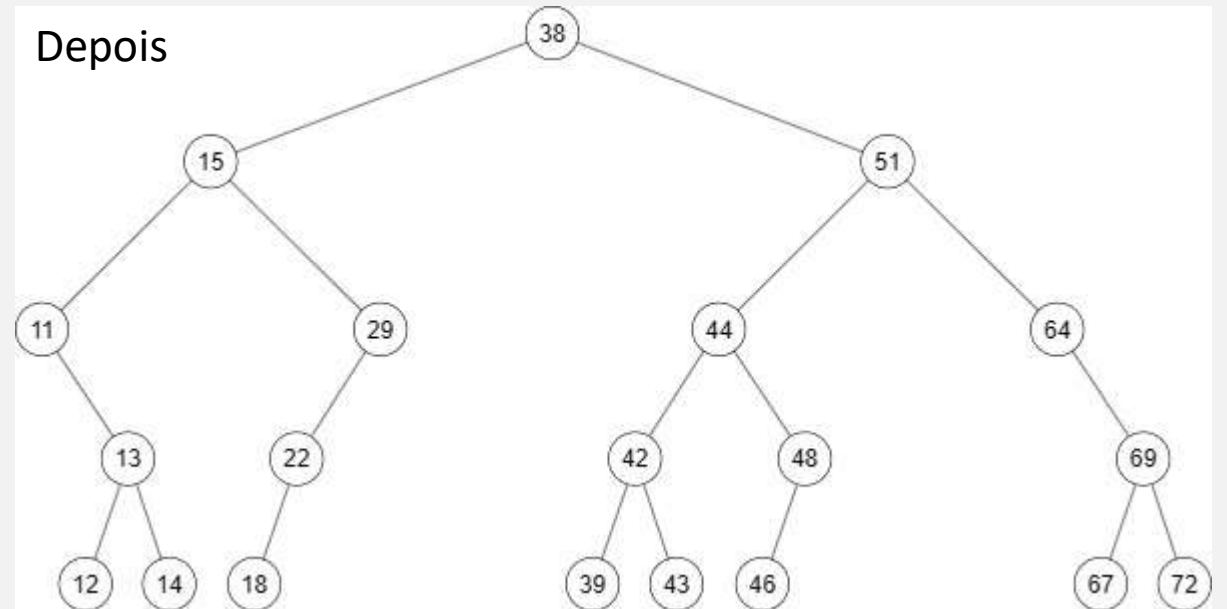
○ REMOÇÃO

- Remoção de nó folha: simplesmente removemos o nó.
- Exemplo: remoção do nó 32;

Antes



Depois

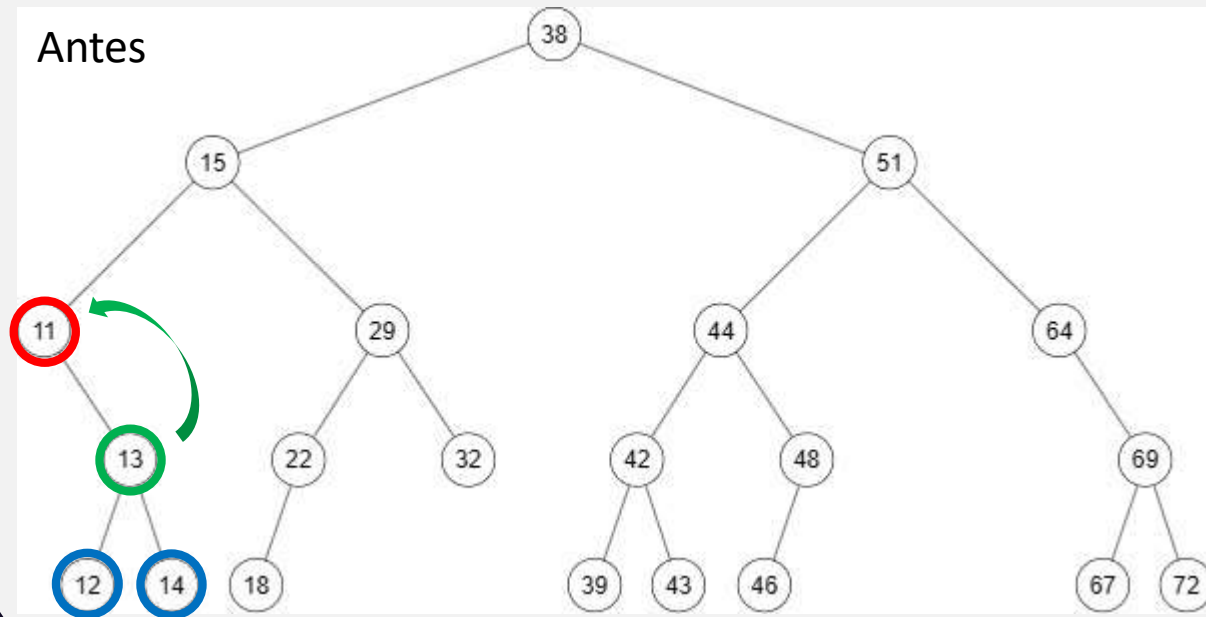


Árvores Binárias de Busca

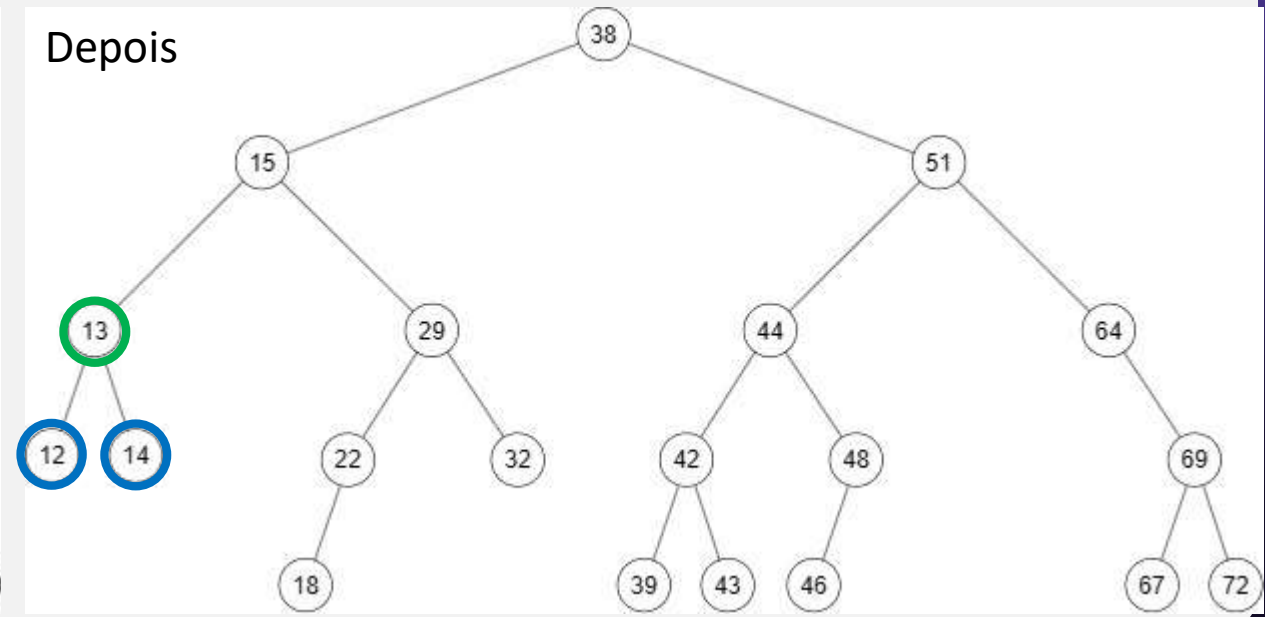
○ REMOÇÃO

- Remoção de nó com apenas 1 filho: o filho (e seus descendentes) sobem 1 nível, ocupando o lugar de seu pai (que será removido).
- Exemplo: remoção do nó 11.

Antes



Depois



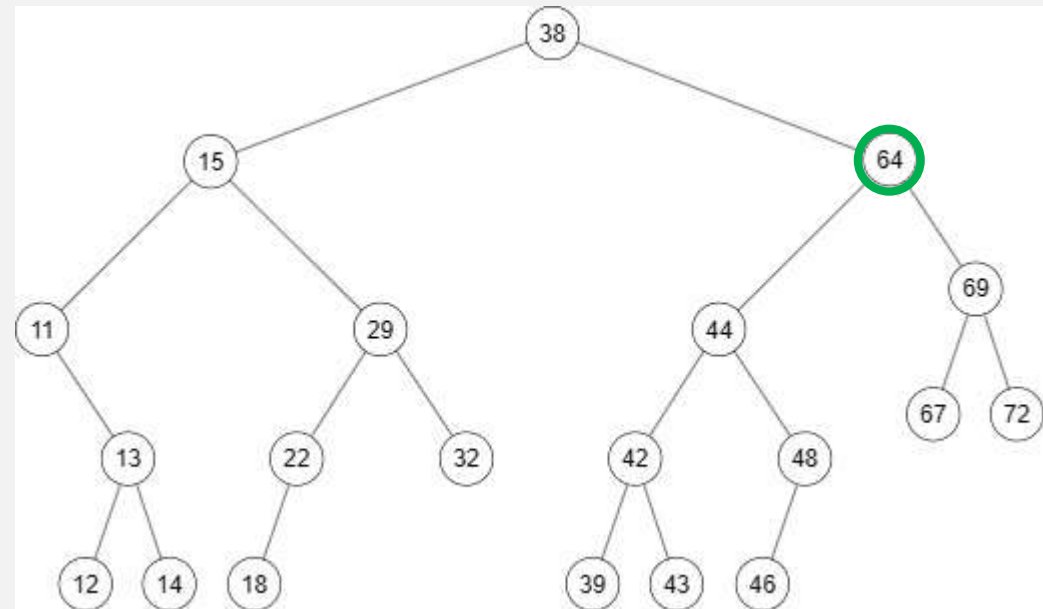
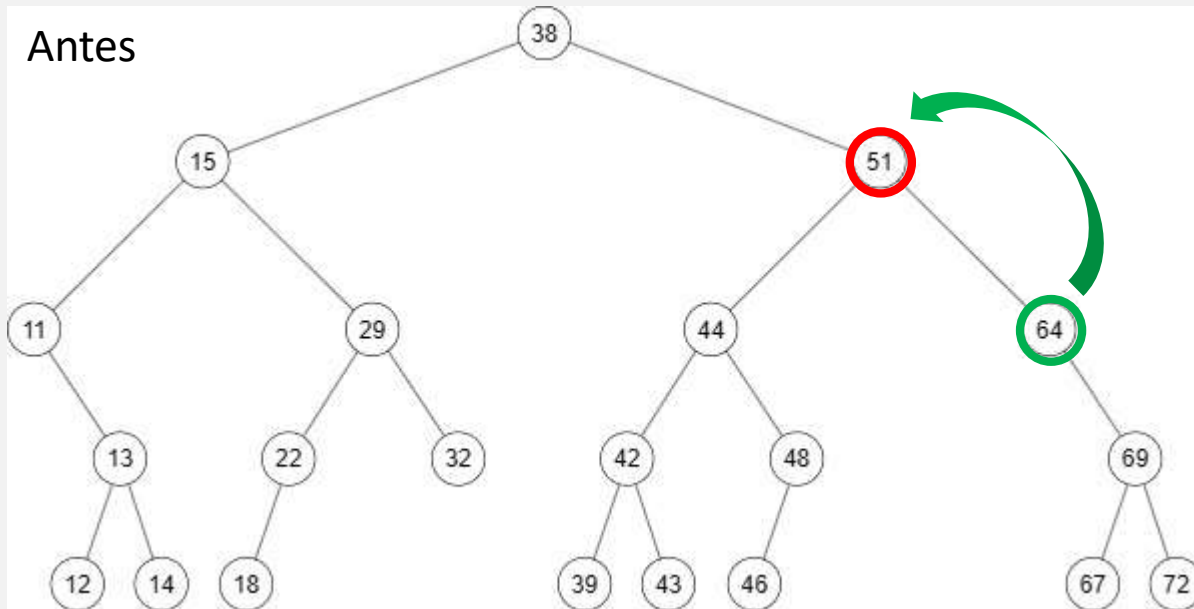
Árvores Binárias de Busca

○ REMOÇÃO

- Remoção de nós com 2 filhos: Precisamos encontrar um substituto para o nó removido.
 - Opção "subir o **sucedor**" (menor valor da subárvore direita).
 - Exemplo: remoção do nó 51.

O sucessor é o menor valor da subárvore direita de 51, ou seja, 64.

Antes



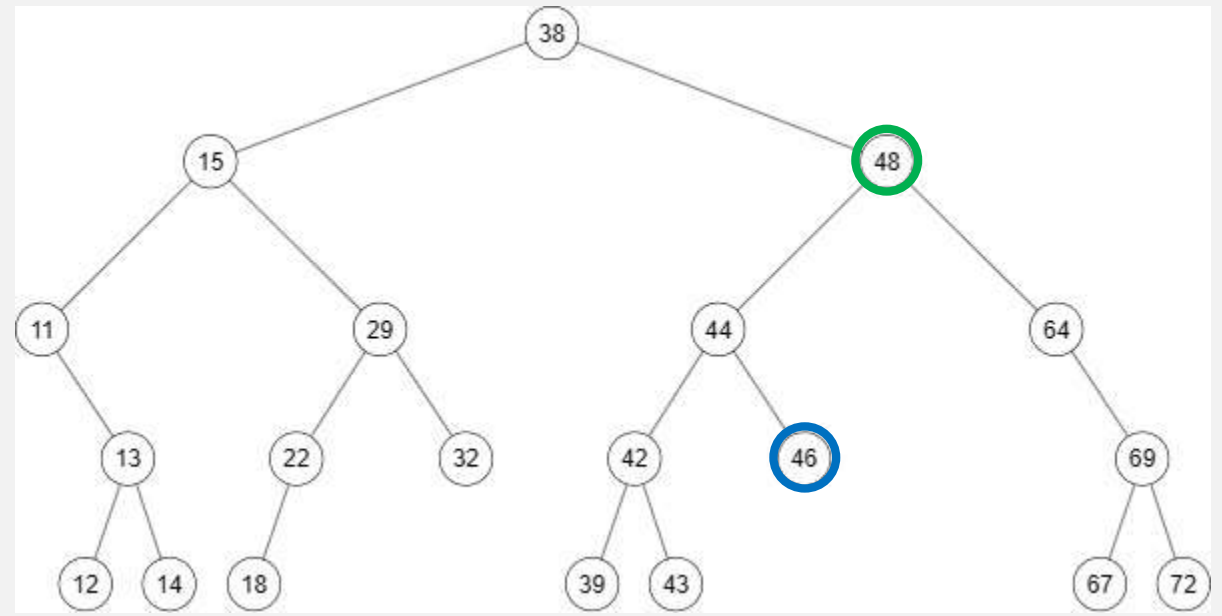
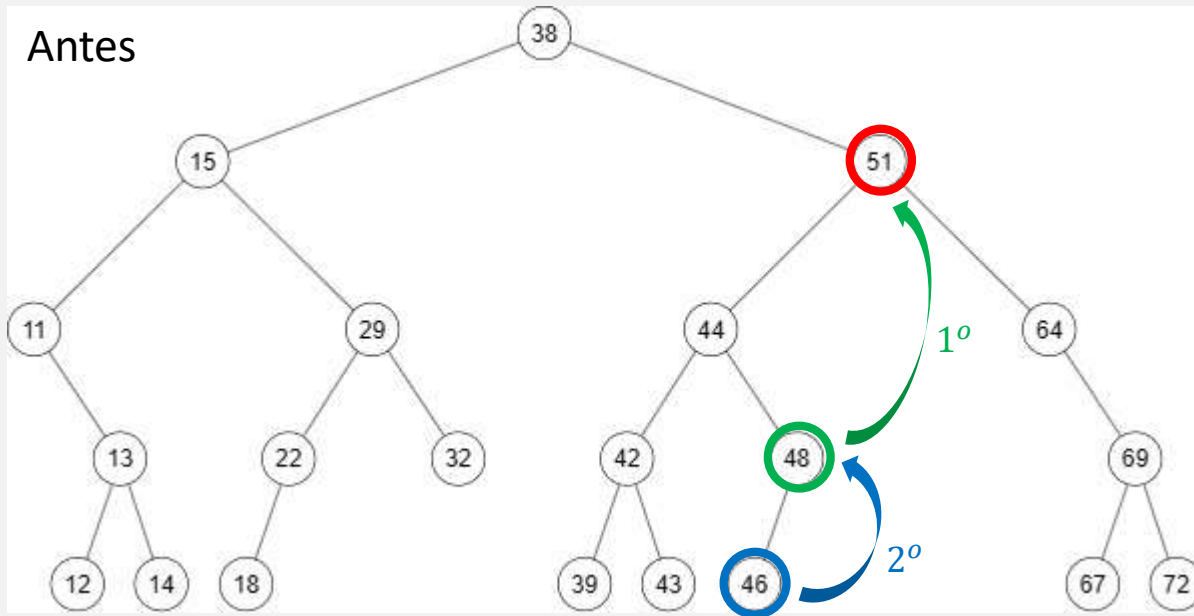
Árvores Binárias de Busca

○ REMOÇÃO

- Remoção de nós com 2 filhos: Precisamos encontrar um substituto para o nó removido.
 - Opção "subir o **antecessor**" (maior valor da subárvore esquerda).
 - Exemplo: remoção do nó 51.

O antecessor é o maior valor da subárvore esquerda de 51, ou seja, 48.

Antes



Implementação

Árvores Binárias de Busca

- IMPLEMENTAÇÃO

- BST (Binary Search Tree = Árvore Binária de Busca).
 - Estrutura de dados hierárquica.
 - Cada nó contém:
 - Chave (valor) para ser utilizada na Lei de Formação, por exemplo: Nome, CPF etc.
 - A chave pode ser um atributo de uma struct ou classe que permita hierarquia de $>$, $=$ e $<$.
 - Não é necessariamente uma árvore que guarda inteiros somente.
 - Ponteiro para filho esquerdo.
 - Ponteiro para filho direito.
 - Regras fundamentais:
 - Valores da chave menores ficam à esquerda.
 - Valores da chave maiores ficam à direita.

Árvores Binárias de Busca

○ IMPLEMENTAÇÃO

○ Representação de um NÓ da árvore:

○ Estrutura em C contendo:

- A chave (exemplo: um inteiro)
- Dois ponteiros para as sub-árvores:
 - Esquerda e
 - Direita

○ Representação de uma ÁRVORE:

- Através de um ponteiro para o nó raiz.

```
typedef struct tNo {
    int chave;
    struct tNo * esquerda;
    struct tNo * direita;
} tNo;

// Função para criar um elemento da árvore
tNo* criaNo(int chave) {
    tNo* novo = (tNo*) malloc(sizeof(tNo));
    novo->chave = chave;
    novo->esquerda = NULL;
    novo->direita = NULL;
    return novo;
}

// Função para inserir um elemento na árvore
// Se o nó existe (chave igual), retorna o nó existente.
// Se não existe, cria um novo nó e insere na árvore.
// Retorna o ponteiro para o nó raiz.
tNo* insere(tNo* raiz, int chave) {
    if (raiz == NULL) {
        raiz = criaNo(chave);
        return raiz;
    }
    if (chave < raiz->chave) {
        raiz->esquerda = insere(raiz->esquerda, chave);
    } else if (chave > raiz->chave) {
        raiz->direita = insere(raiz->direita, chave);
    }
    return raiz;
}

// Função para remover um elemento da árvore
// Se o nó existe, remove o nó e retorna o nó raiz.
// Se não existe, retorna NULL.
tNo* remove(tNo* raiz, int chave) {
    if (raiz == NULL) {
        return NULL;
    }
    if (chave < raiz->chave) {
        raiz->esquerda = remove(raiz->esquerda, chave);
    } else if (chave > raiz->chave) {
        raiz->direita = remove(raiz->direita, chave);
    } else {
        // Encontramos o nó a ser removido
        if (raiz->esquerda == NULL || raiz->direita == NULL) {
            // Caso 1: O nó a ser removido é uma folha
            tNo* temp = raiz->esquerda;
            if (temp == NULL) temp = raiz->direita;
            free(raiz);
            return temp;
        } else {
            // Caso 2: O nó a ser removido tem filhos
            tNo* temp = raiz->esquerda;
            while (temp->direita != NULL) {
                temp = temp->direita;
            }
            // Substitui o nó a ser removido pelo seu sucessor
            raiz->chave = temp->chave;
            raiz->esquerda = temp->esquerda;
            raiz->direita = temp->direita;
            free(temp);
        }
    }
    return raiz;
}

// Função para imprimir a árvore em ordem
void imprime(tNo* raiz) {
    if (raiz == NULL) return;
    imprime(raiz->esquerda);
    printf("%d ", raiz->chave);
    imprime(raiz->direita);
}

// Função para verificar se a árvore está vazia
int vazia(tNo* raiz) {
    return raiz == NULL;
}

// Função para buscar um elemento na árvore
// Retorna o ponteiro para o nó encontrado, ou NULL se não encontrado.
tNo* busca(tNo* raiz, int chave) {
    if (raiz == NULL) return NULL;
    if (raiz->chave == chave) return raiz;
    if (chave < raiz->chave) return busca(raiz->esquerda, chave);
    if (chave > raiz->chave) return busca(raiz->direita, chave);
    return NULL;
}

// Função para calcular o tamanho da árvore
int tamanho(tNo* raiz) {
    if (raiz == NULL) return 0;
    return 1 + tamanho(raiz->esquerda) + tamanho(raiz->direita);
}

// Função para calcular a altura da árvore
int altura(tNo* raiz) {
    if (raiz == NULL) return 0;
    return 1 + max(altura(raiz->esquerda), altura(raiz->direita));
}

// Função para verificar se a árvore é balanceada
int balanceada(tNo* raiz) {
    if (raiz == NULL) return 1;
    int esquerda = altura(raiz->esquerda);
    int direita = altura(raiz->direita);
    return (abs(esquerda - direita) <= 1) && balanceada(raiz->esquerda) && balanceada(raiz->direita);
}

// Função para verificar se a árvore é uma árvore de busca binária
int eabb(tNo* raiz) {
    if (raiz == NULL) return 1;
    int esquerda = busca(raiz->esquerda, raiz->chave);
    int direita = busca(raiz->direita, raiz->chave);
    return (esquerda == NULL && direita == NULL) && eabb(raiz->esquerda) && eabb(raiz->direita);
}

// Função para verificar se a árvore é uma árvore de busca binária
int eabb(tNo* raiz) {
    if (raiz == NULL) return 1;
    int esquerda = busca(raiz->esquerda, raiz->chave);
    int direita = busca(raiz->direita, raiz->chave);
    return (esquerda == NULL && direita == NULL) && eabb(raiz->esquerda) && eabb(raiz->direita);
}
```

Node

"data"

"key" (uma das variáveis de 'data')

Node * Left;
Node * Right;

Tree

Node * root;

Árvores Binárias de Busca

○ IMPLEMENTAÇÃO

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4 #include <string.h>
5
6 typedef struct Node {
7     int key;
8     struct Node * Left;
9     struct Node * Right;
10 } Node;
11
12 typedef struct {
13     Node * root;
14 } Tree;
15
16 // Função para criar a árvore
17 Tree * CreateTree() {
18     Tree * newTree = (Tree *)malloc(sizeof(Tree));
19     newTree->root = NULL;
20     return newTree;
21 }
```

```
23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { // Se está disponível, insere e retorna.
26         Node * novoNo = (Node *)malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     // Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); // Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); // Recursividade.
38     }
39     return no; // Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }
```

Árvores Binárias de Busca

○ IMPLEMENTAÇÃO

```
46 // Função para imprimir a árvore em ordem
47 void PrintNode(Node * no, int level) {
48     if (no != NULL) {
49         // Criação do texto de indentação
50         char txt[100]; // Tamanho suficiente para a indentação
51         txt[0] = '\0'; // Inicializa a string
52
53         for (int c = 0; c < level - 1; c++) {
54             strcat(txt, " "); // Adiciona ' ' para cada nível
55         }
56         if (level > 0) strcat(txt, "|.."); // Adiciona '...' para cada nível
57
58         // Imprime o nó atual
59         printf("%s%d\n", txt, no->key);
60
61         // Chamada recursiva para o filho Left
62         PrintNode(no->Left, level + 1);
63
64         // Chamada recursiva para o filho Right
65         PrintNode(no->Right, level + 1);
66     }
67 }
68
69 void Print(Tree * Tree) {
70     PrintNode(Tree->root, 0);
71     printf("\n");
72 }
```

```
74 // Função para verificar se a árvore está vazia
75 int IsEmpty(Tree * Tree) {
76     return Tree->root == NULL;
77 }
78
79 // Função para buscar um elemento na árvore
80 bool SearchNode(Node * no, int value) {
81     if (no == NULL) {
82         return false; // não encontrado
83     }
84     if (no->key == value) {
85         return true; // encontrado
86     }
87     if (value < no->key) {
88         return SearchNode(no->Left, value); //Recursividade.
89     }
90     else {
91         return SearchNode(no->Right, value); //Recursividade.
92     }
93 }
94
95 bool Search(Tree * Tree, int value) {
96     return SearchNode(Tree->root, value);
97 }
98
99 // Função para liberar a memória da árvore
100 void FreeNode(Node * no) {
101     if (no != NULL) {
102         FreeNode(no->Left); //Recursividade.
103         FreeNode(no->Right); //Recursividade.
104         free(no);
105     }
106 }
107
108 void Free(Tree * Tree) {
109     FreeNode(Tree->root);
110     free(Tree);
111 }
```

Árvores Binárias de Busca

○ IMPLEMENTAÇÃO

```
113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134
135         // Nó com dois filhos: obter o menor no da subárvore da Right
136         Node * temp = no->Right;
137         while (temp && temp->Left != NULL) {
138             temp = temp->Left;
139         }
140
141         no->key = temp->key; // Copiar o value do nó
142         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
143     }
144     return no;
145 }
146
147 //Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
148 void Remove(Tree * Tree, int value) {
149     Tree->root = RemoveNode(Tree->root, value);
150 }
```

Árvores Binárias de Busca

- MAIN()

```
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }
```

```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

16 // Função para criar a árvore
17 Tree * CreateTree() {
18     Tree * newTree = (Tree * )malloc(sizeof(Tree));
19     newTree->root = NULL;
20     return newTree;
21 }

```

⓪

```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

16 // Função para criar a árvore
17 Tree * CreateTree() {
18     Tree * newTree = (Tree *)malloc(sizeof(Tree));
19     newTree->root = NULL;
20     return newTree;
21 }

```

Ⓝ

```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

16 // Função para criar a árvore
17 Tree * CreateTree() {
18     Tree * newTree = (Tree * )malloc(sizeof(Tree));
19     newTree->root = NULL;
20     return newTree;
21 }

```

null


```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recurividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recurividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```

⓪ null

```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```

null

```

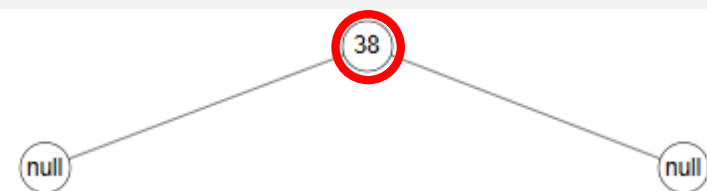
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

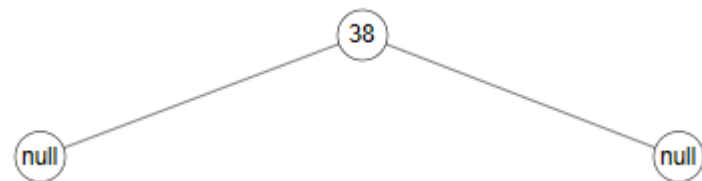
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

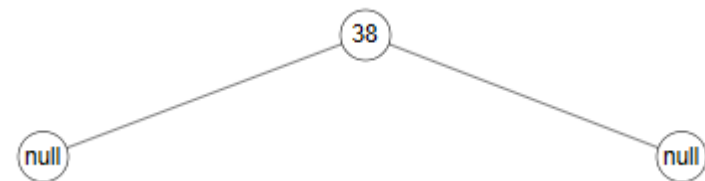
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

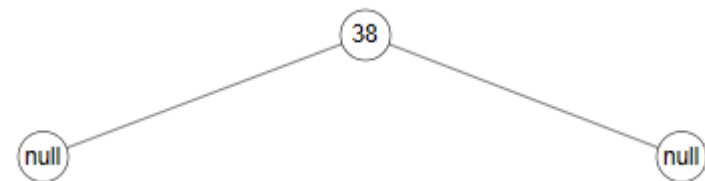
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```




```

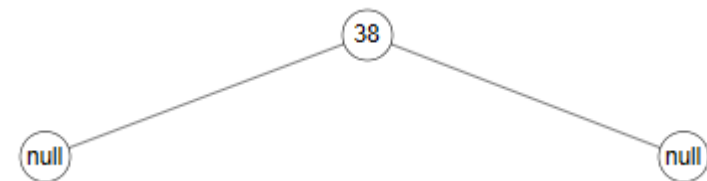
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

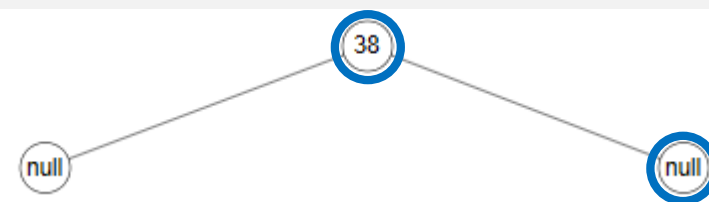
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node * )malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```




```

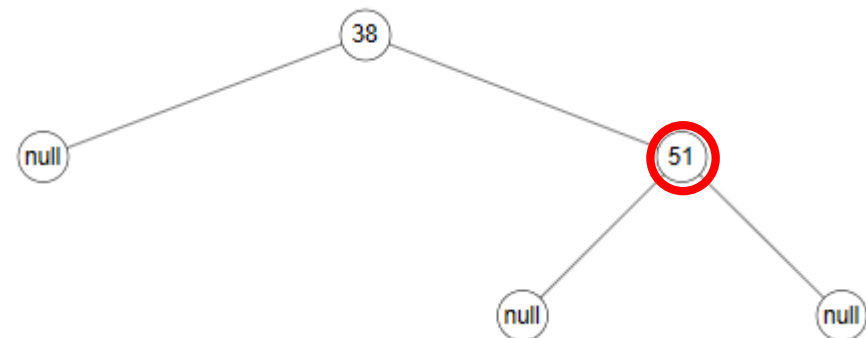
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

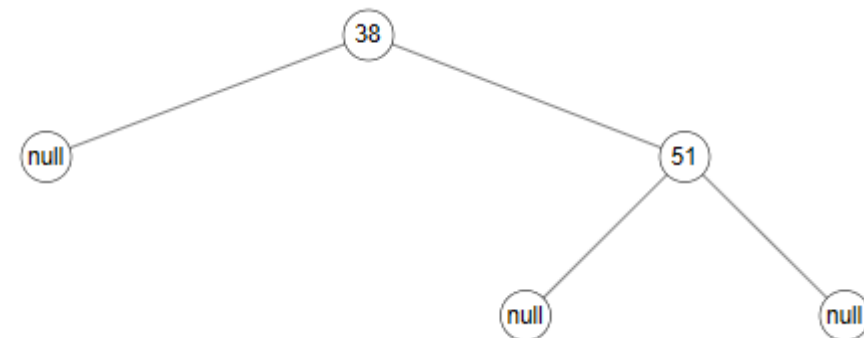
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

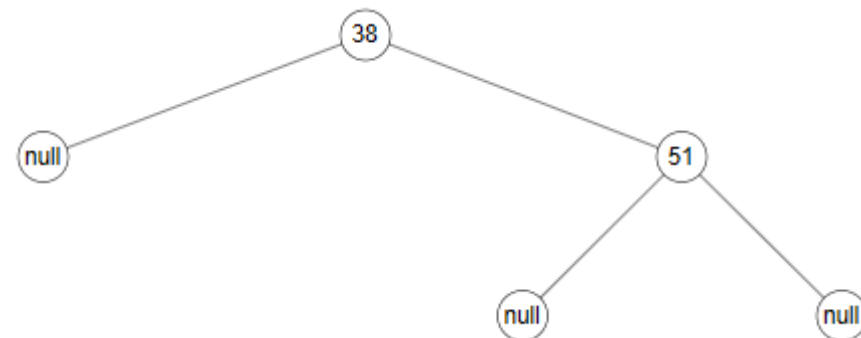
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

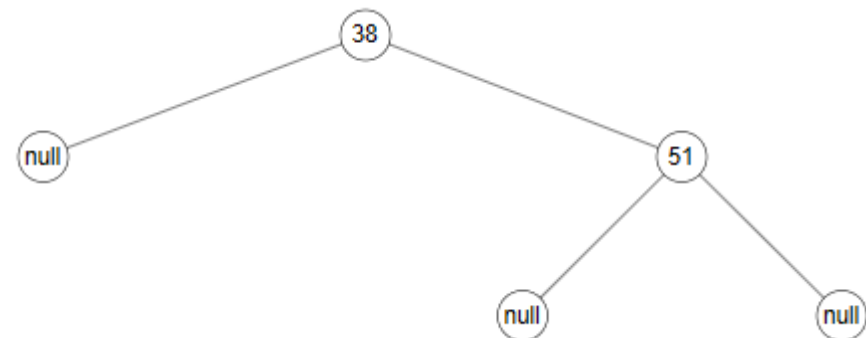
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

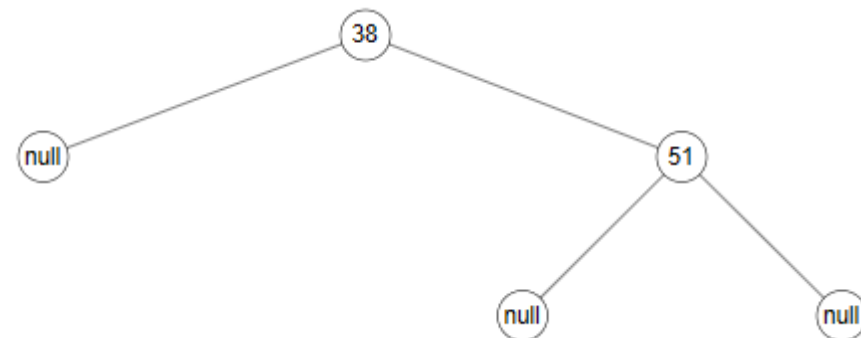
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

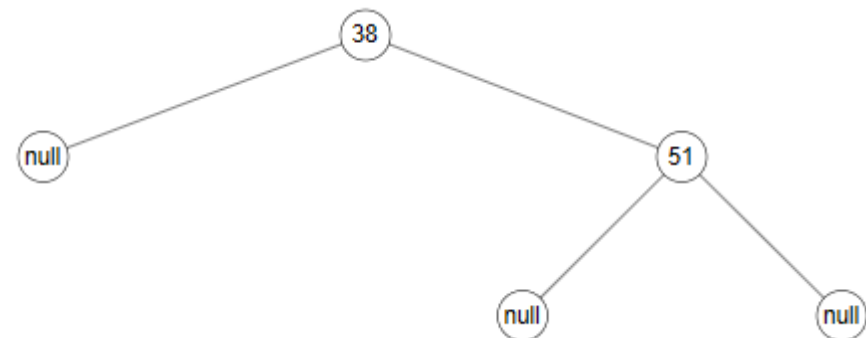
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```




```

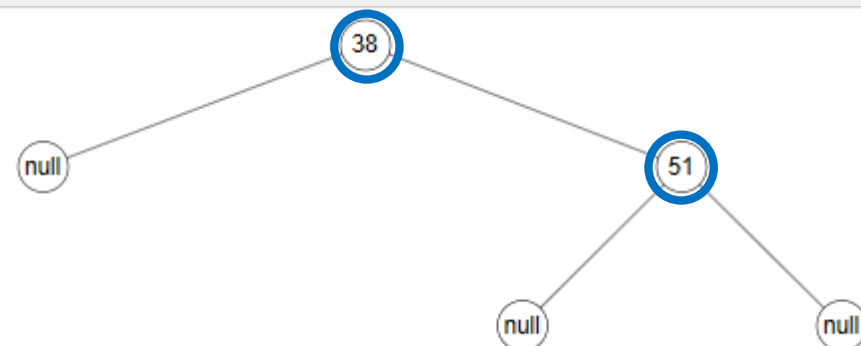
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

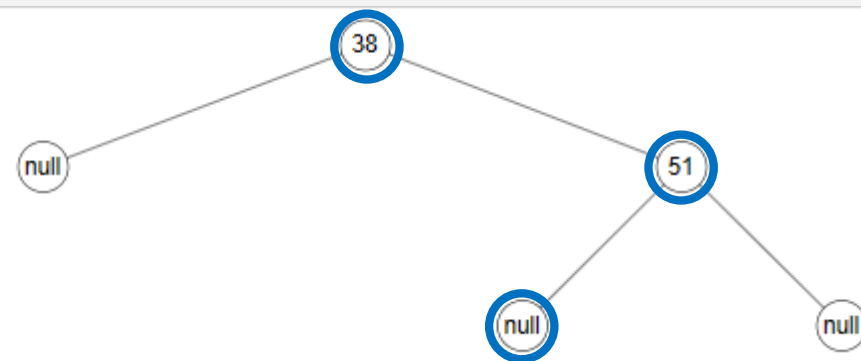
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontra");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```




```

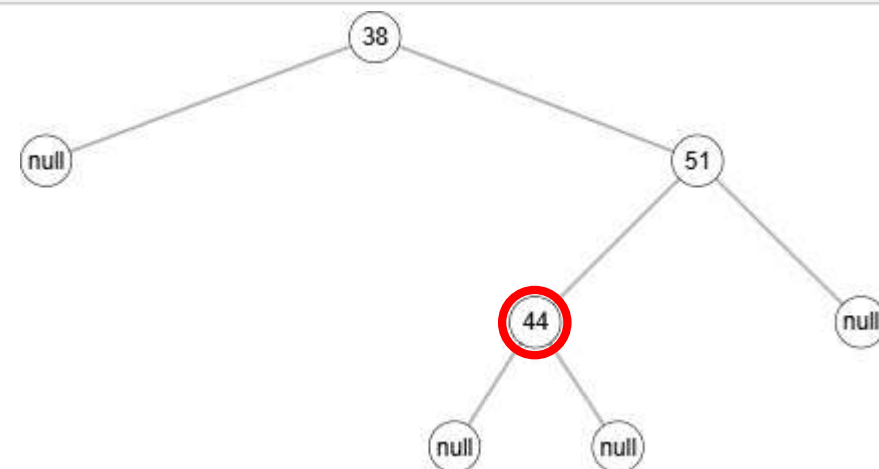
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

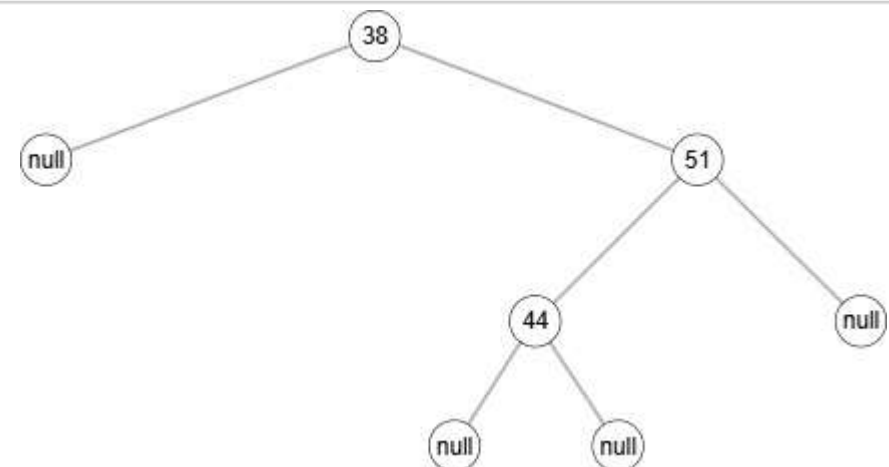
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

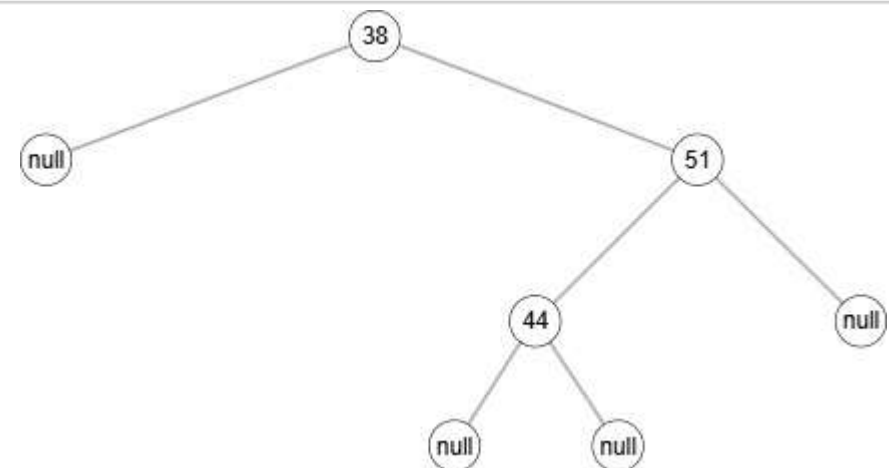
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

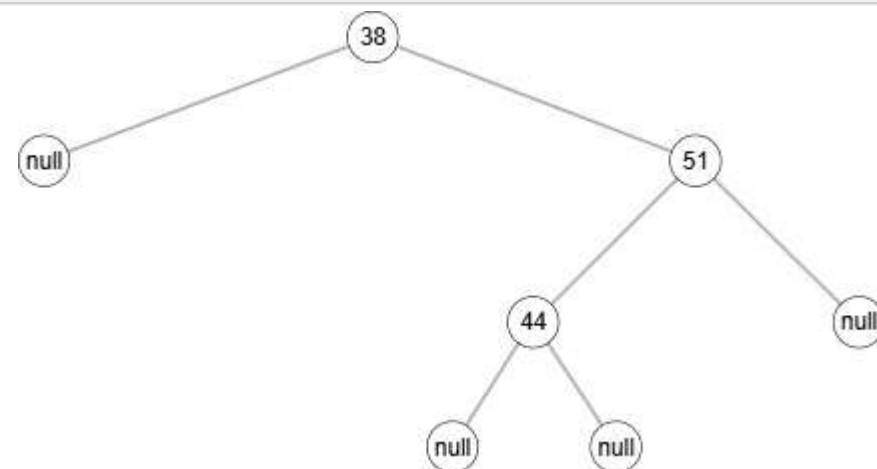
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

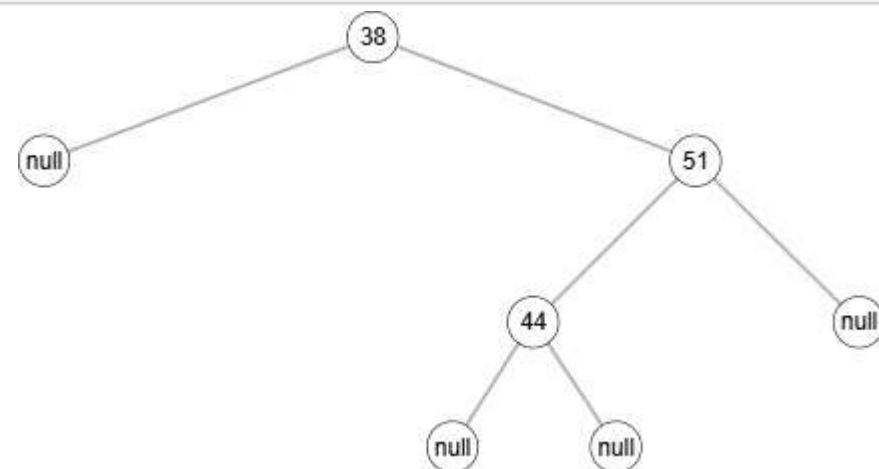
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```




```

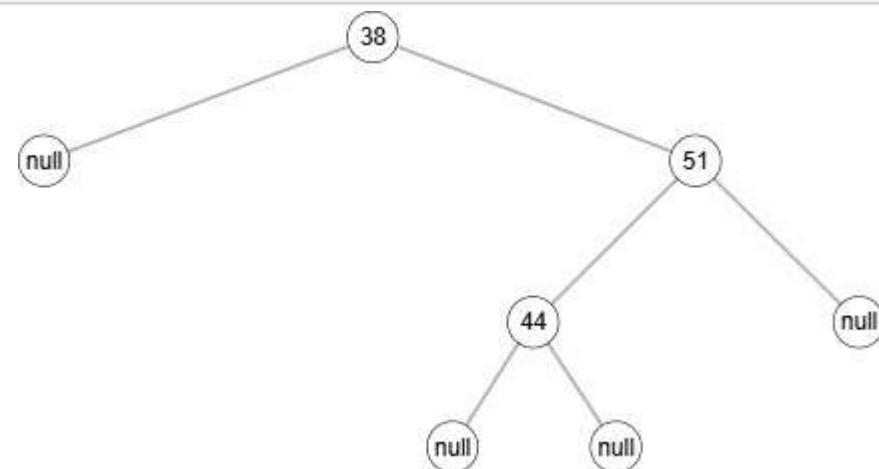
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

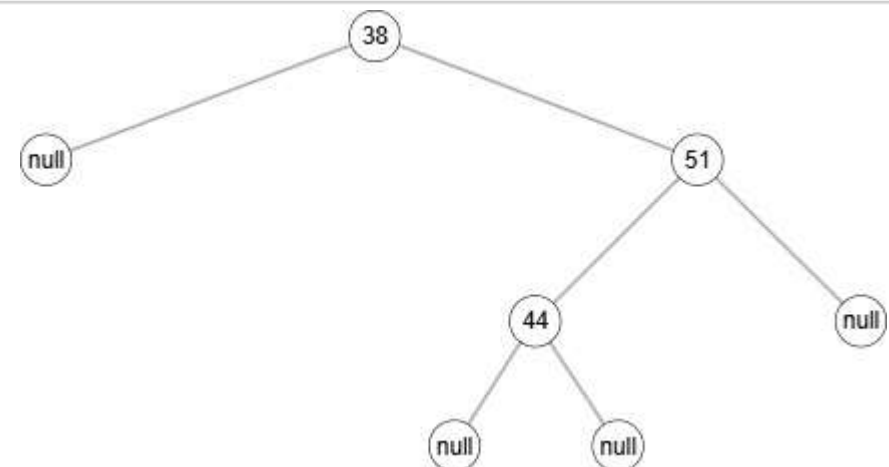
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

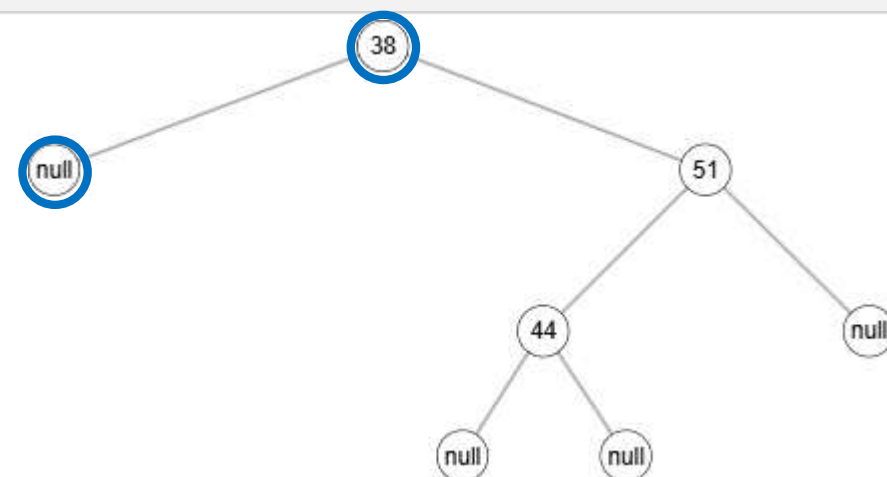
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```




```

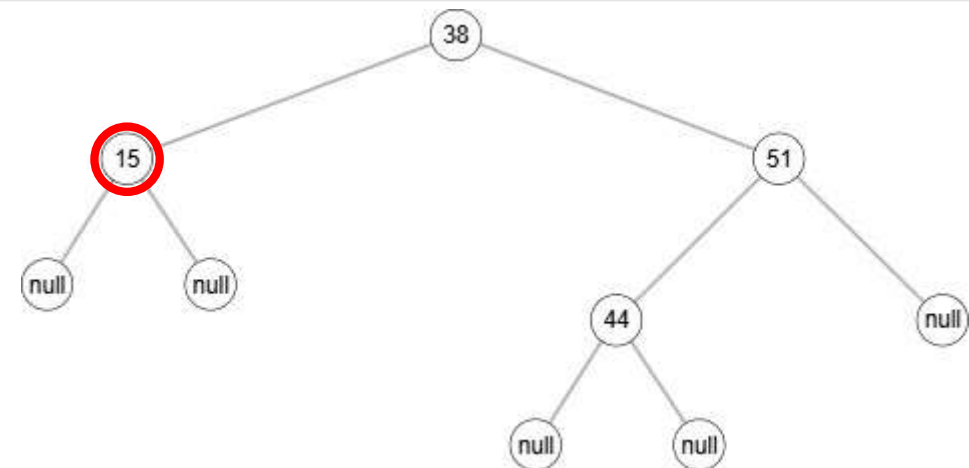
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

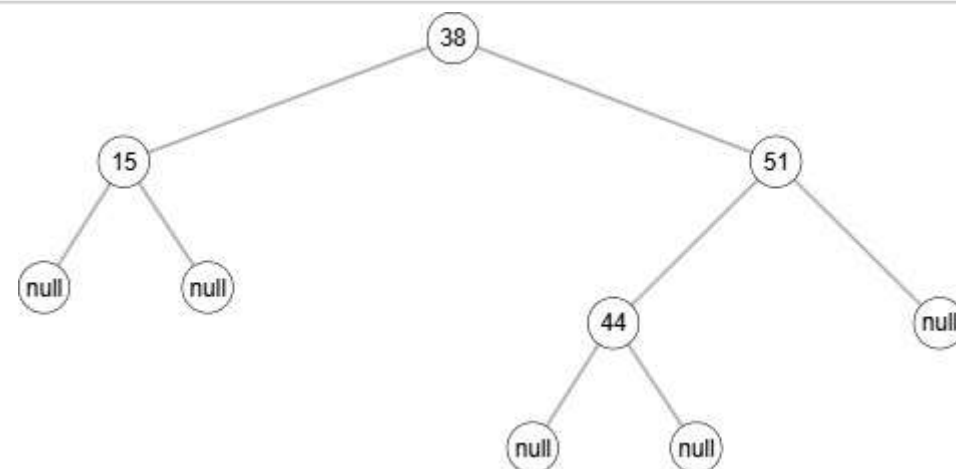
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

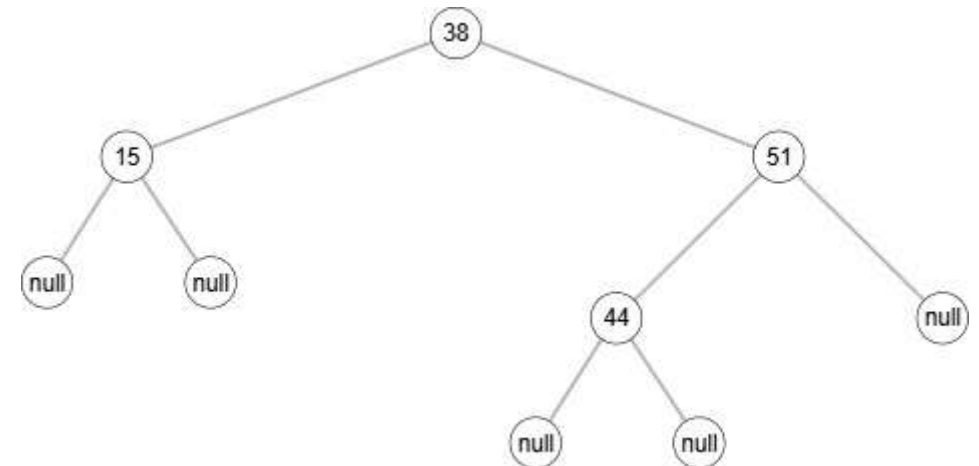
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

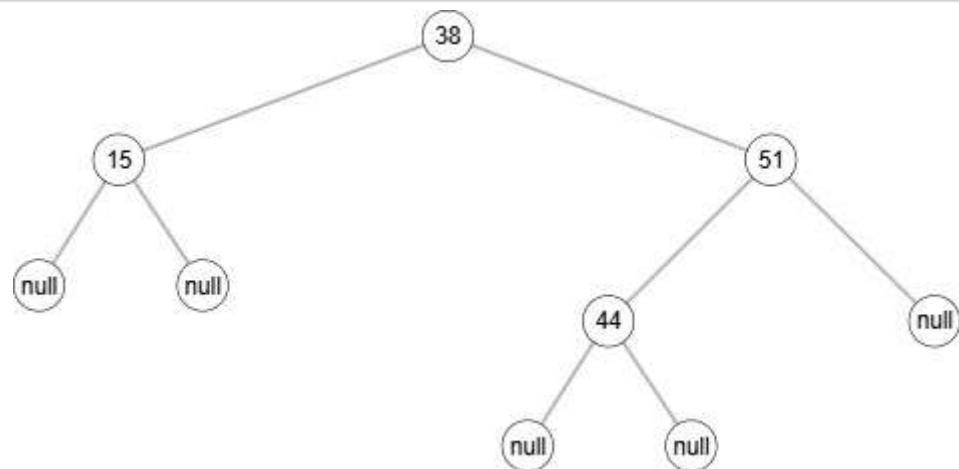
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

```



```

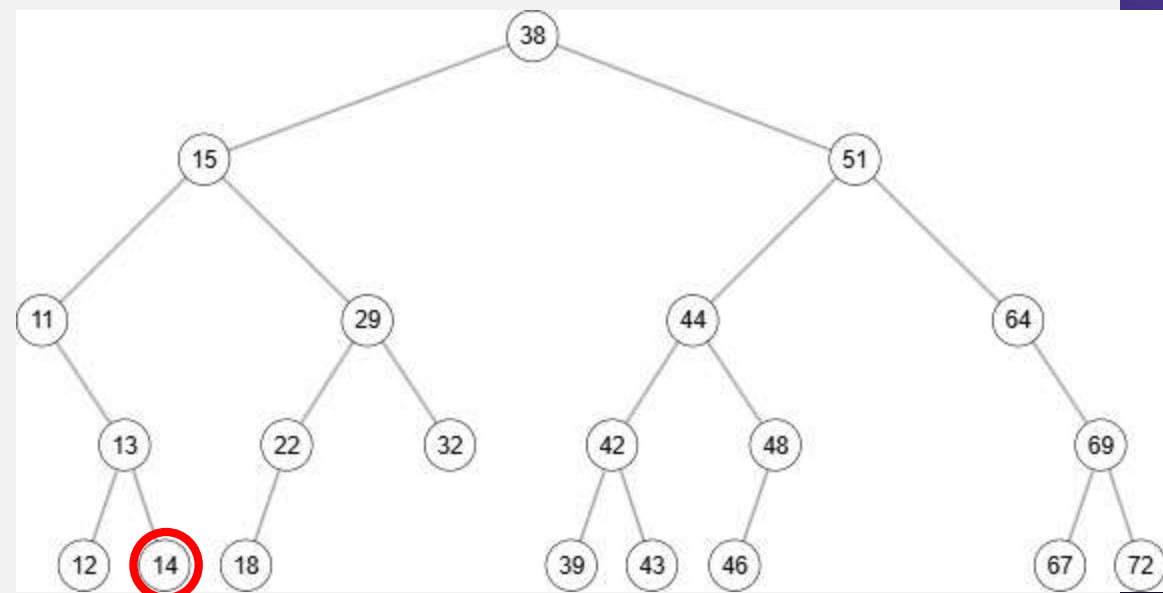
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

23 // Função para inserir um elemento na árvore
24 Node * InsertNode(Node * no, int value) {
25     if (no == NULL) { //Se está disponível, insere e retorna.
26         Node * novoNo = (Node *) malloc(sizeof(Node));
27         novoNo->key = value;
28         novoNo->Left = NULL;
29         novoNo->Right = NULL;
30         return novoNo;
31     }
32     //Se já tem 2 filhos, a busca continua...
33     if (value < no->key) {
34         no->Left = InsertNode(no->Left, value); //Recursividade.
35     }
36     else if (value > no->key) {
37         no->Right = InsertNode(no->Right, value); //Recursividade.
38     }
39     return no; //Vai retornar o mesmo nó caso o value seja igual.
40 }
41
42 void Insert(Tree * Tree, int value) {
43     Tree->root = InsertNode(Tree->root, value);
44 }

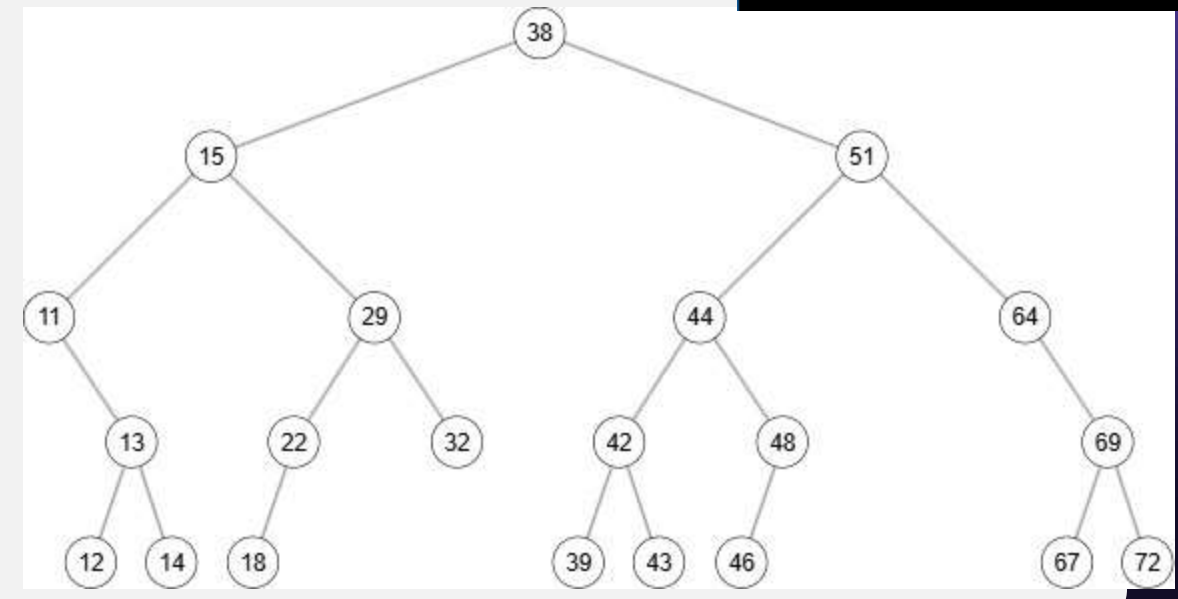
```



Elementos da árvore:
 38
 |..15
 |..11
 |..13
 |..12
 |..14
 |..29
 |..22
 |..18
 |..32
 |..51
 |..44
 |..42
 |..39
 |..43
 |..48
 |..46
 |..64
 |..69
 |..67
 |..72

```

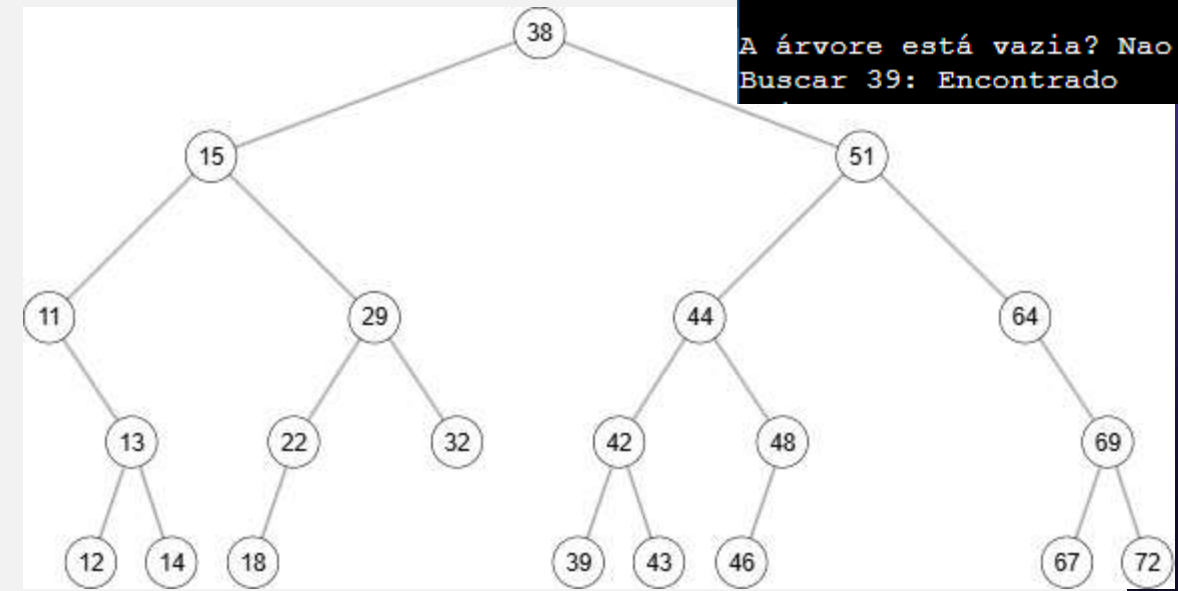
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }
  
```



```

Elementos da árvore:
38
|..15
|..11
|..13
|..12
|..14
|..29
|..22
|..18
|..32
|..51
|..44
|..42
|..39
|..43
|..48
|..46
|..64
|..69
|..67
|..72
    
```

A árvore está vazia? Nao
 Buscar 39: Encontrado



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }
    
```

```

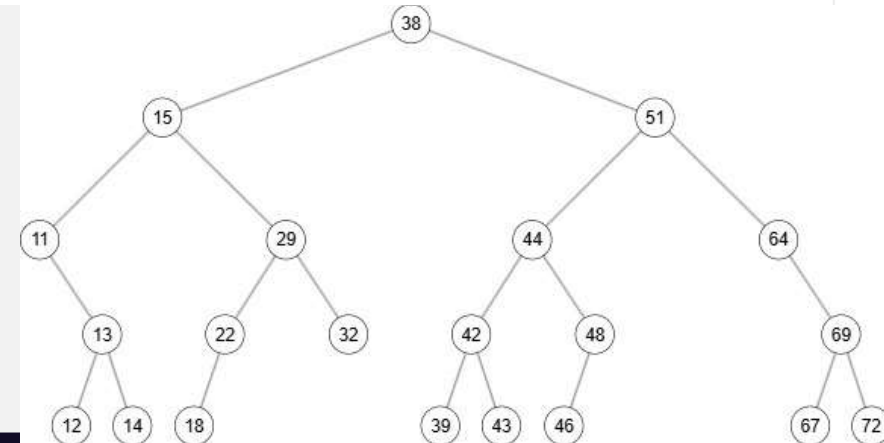
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134
135         // Nó com dois filhos: obter o menor no da subárvore da Right
136         Node * temp = no->Right;
137         while (temp && temp->Left != NULL) {
138             temp = temp->Left;
139         }
140
141         no->key = temp->key; // Copiar o value do nó
142         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
143     }
144     return no;
145 }
146
147 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
148 void Remove(Tree * Tree, int value) {
149     Tree->root = RemoveNode(Tree->root, value);
150 }

```




```

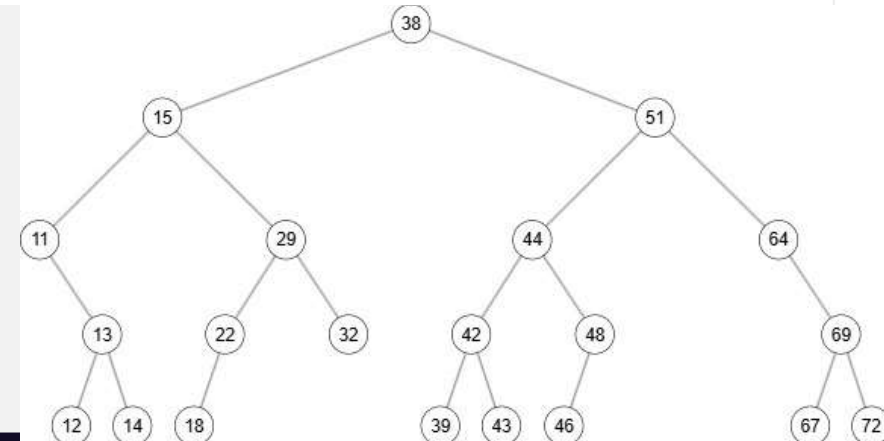
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134
135         // Nó com dois filhos: obter o menor no da subárvore da Right
136         Node * temp = no->Right;
137         while (temp && temp->Left != NULL) {
138             temp = temp->Left;
139         }
140
141         no->key = temp->key; // Copiar o value do nó
142         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
143     }
144     return no;
145 }
146
147 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
148 void Remove(Tree * Tree, int value) {
149     Tree->root = RemoveNode(Tree->root, value);
150 }

```



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

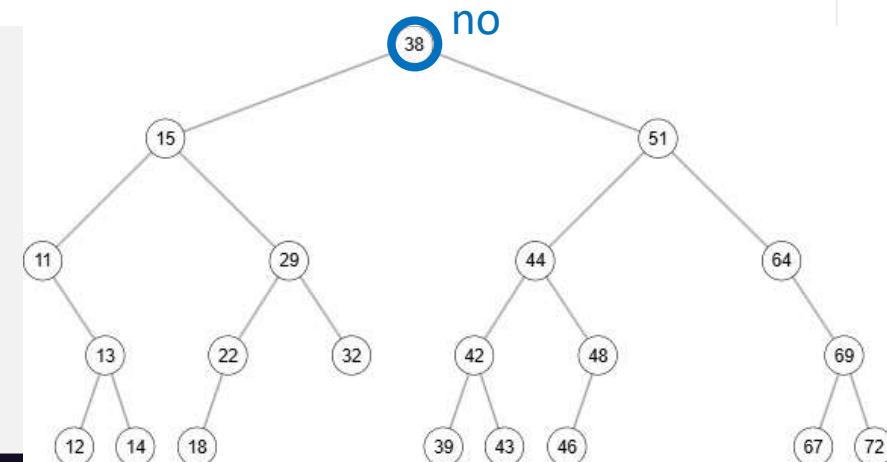
```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134
135         // Nó com dois filhos: obter o menor no da subárvore da Right
136         Node * temp = no->Right;
137         while (temp && temp->Left != NULL) {
138             temp = temp->Left;
139         }
140
141         no->key = temp->key; // Copiar o value do nó
142         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
143     }
144     return no;
145 }
146
147 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
148 void Remove(Tree * Tree, int value) {
149     Tree->root = RemoveNode(Tree->root, value);
150 }

```

no 38
right 51



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

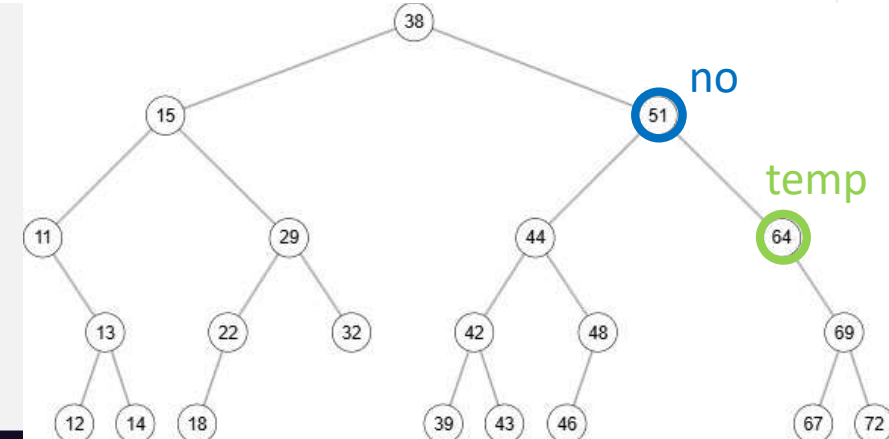
```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134         // Nó com dois filhos: obter o menor no da subárvore da Right
135         Node * temp = no->Right;
136         while (temp && temp->Left != NULL) {
137             temp = temp->Left;
138         }
139
140         no->key = temp->key; // Copiar o value do nó
141         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
142     }
143     return no;
144 }
145
146 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
147 void Remove(Tree * Tree, int value) {
148     Tree->root = RemoveNode(Tree->root, value);
149 }

```

no 38
right 51

no 51
temp 64



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

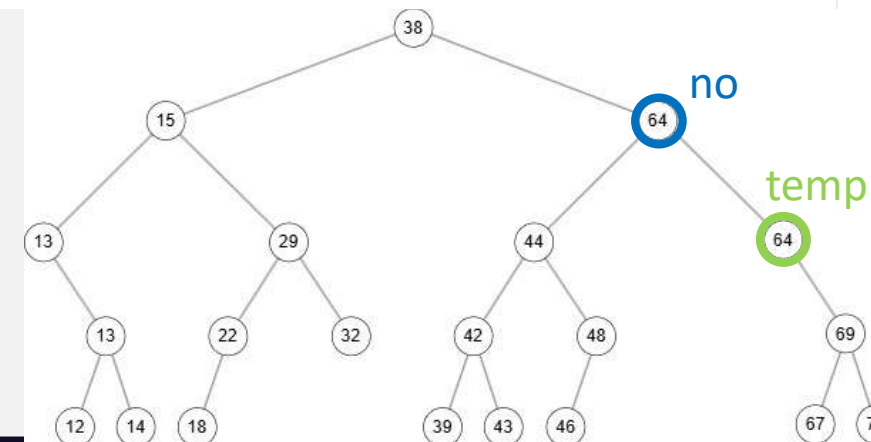
```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134         // Nó com dois filhos: obter o menor no da subárvore da Right
135         Node * temp = no->Right;
136         while (temp && temp->Left != NULL) {
137             temp = temp->Left;
138         }
139         no->key = temp->key; // Copiar o value do nó
140         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
141     }
142     return no;
143 }
144
145 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
146 void Remove(Tree * Tree, int value) {
147     Tree->root = RemoveNode(Tree->root, value);
148 }

```

no 38
right 51

no 64
temp 64*



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

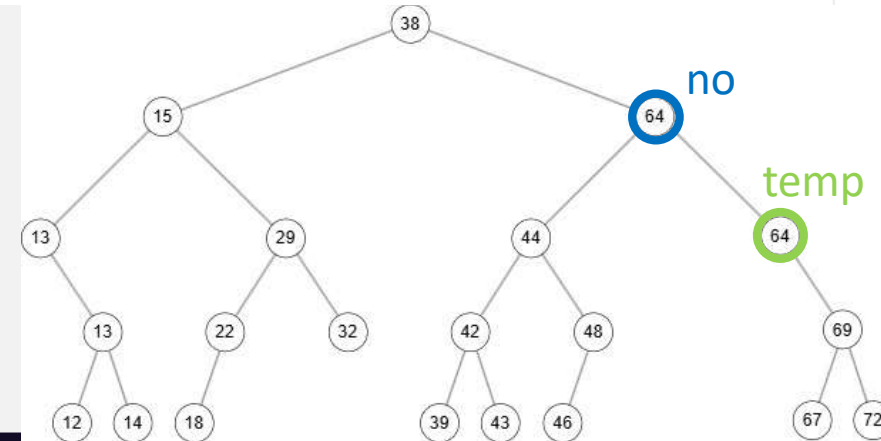
```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134         // Nó com dois filhos: obter o menor no da subárvore da Right
135         Node * temp = no->Right;
136         while (temp && temp->Left != NULL) {
137             temp = temp->Left;
138         }
139         no->key = temp->key; // Copiar o value do nó
140         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
141     }
142     return no;
143 }
144
145 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
146 void Remove(Tree * Tree, int value) {
147     Tree->root = RemoveNode(Tree->root, value);
148 }

```

no 38
right 64

no 64
temp 64*




```

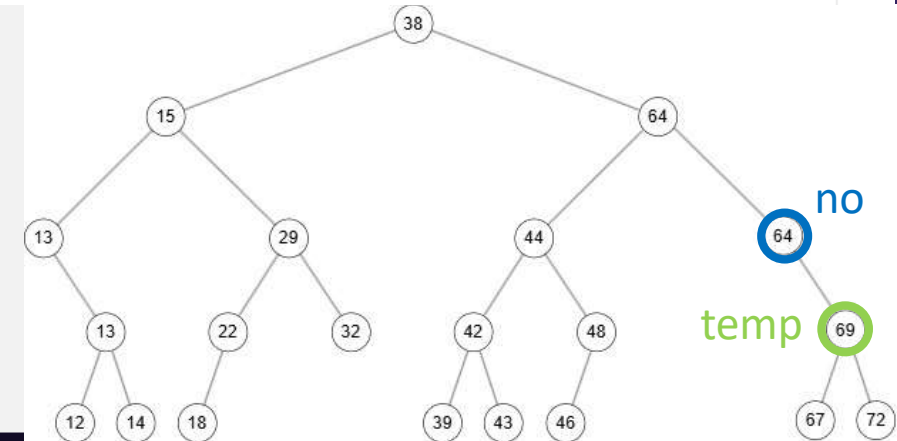
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134         // Nó com dois filhos: obter o menor no da subárvore da Right
135         Node * temp = no->Right;
136         while (temp && temp->Left != NULL) {
137             temp = temp->Left;
138         }
139         no->key = temp->key; // Copiar o value do nó
140         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
141     }
142     return no;
143 }
144
145 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
146 void Remove(Tree * Tree, int value) {
147     Tree->root = RemoveNode(Tree->root, value);
148 }

```



```

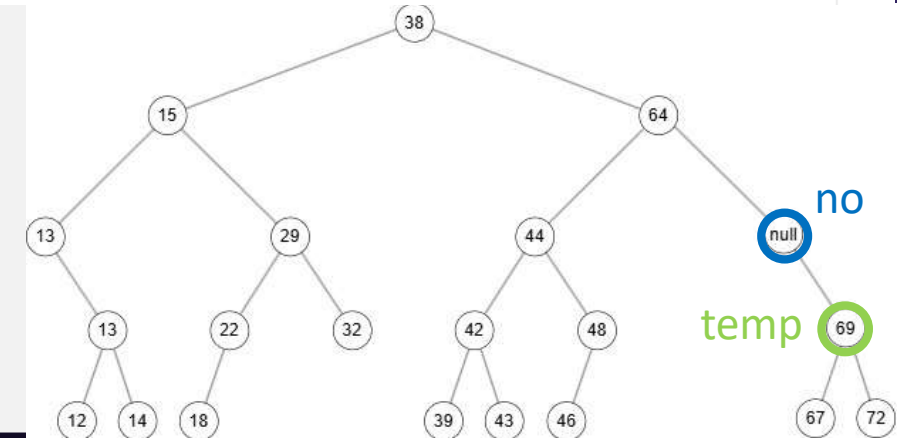
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134         // Nó com dois filhos: obter o menor no da subárvore da Right
135         Node * temp = no->Right;
136         while (temp && temp->Left != NULL) {
137             temp = temp->Left;
138         }
139         no->key = temp->key; // Copiar o value do nó
140         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
141     }
142     return no;
143 }
144
145 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
146 void Remove(Tree * Tree, int value) {
147     Tree->root = RemoveNode(Tree->root, value);
148 }

```



```

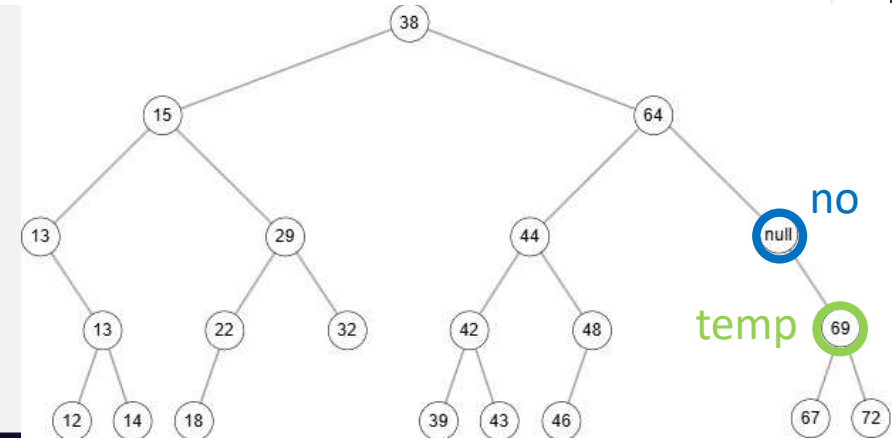
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134         // Nó com dois filhos: obter o menor no da subárvore da Right
135         Node * temp = no->Right;
136         while (temp && temp->Left != NULL) {
137             temp = temp->Left;
138         }
139         no->key = temp->key; // Copiar o value do nó
140         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
141     }
142     return no;
143 }
144
145 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
146 void Remove(Tree * Tree, int value) {
147     Tree->root = RemoveNode(Tree->root, value);
148 }

```




```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

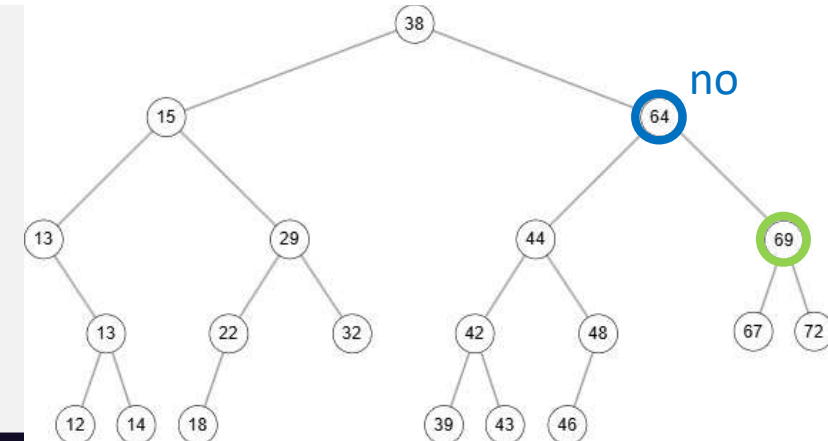
```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134         // Nó com dois filhos: obter o menor no da subárvore da Right
135         Node * temp = no->Right;
136         while (temp && temp->Left != NULL) {
137             temp = temp->Left;
138         }
139         no->key = temp->key; // Copiar o value do nó
140         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
141     }
142     return no;
143 }
144
145 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
146 void Remove(Tree * Tree, int value) {
147     Tree->root = RemoveNode(Tree->root, value);
148 }

```

no 38
right 64

no 64



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

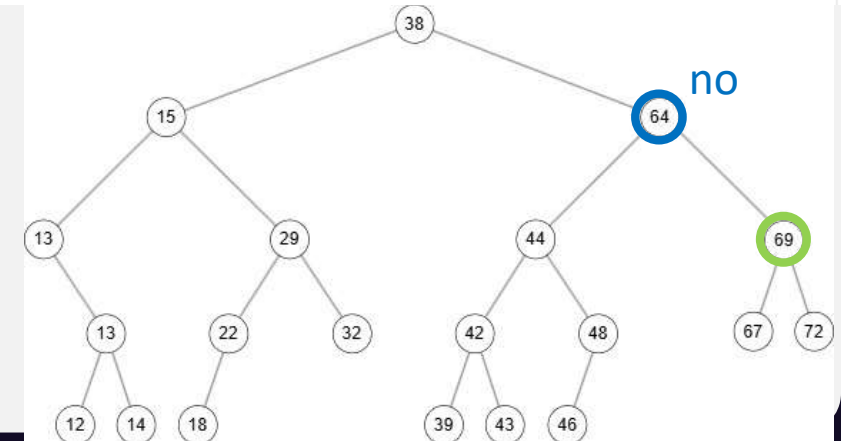
```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134
135         // Nó com dois filhos: obter o menor no da subárvore da Right
136         Node * temp = no->Right;
137         while (temp && temp->Left != NULL) {
138             temp = temp->Left;
139         }
140
141         no->key = temp->key; // Copiar o value do nó
142         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
143     }
144     return no;
145 }
146
147 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
148 void Remove(Tree * Tree, int value) {
149     Tree->root = RemoveNode(Tree->root, value);
150 }

```

no 38
right 64

no 69



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

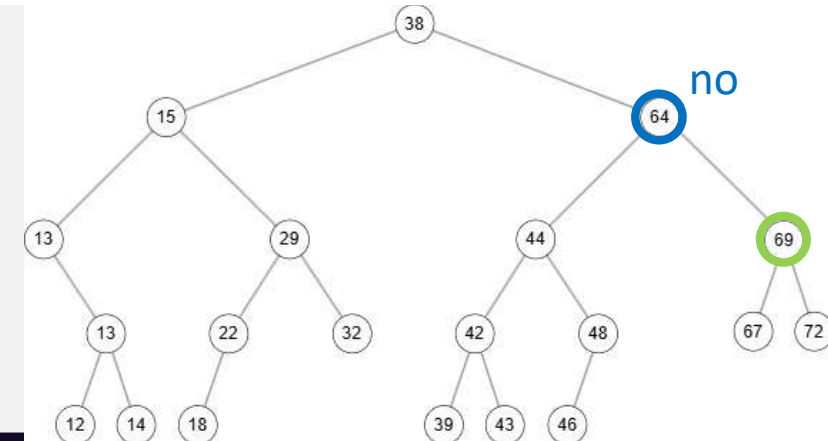
```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134
135         // Nó com dois filhos: obter o menor no da subárvore da Right
136         Node * temp = no->Right;
137         while (temp && temp->Left != NULL) {
138             temp = temp->Left;
139         }
140
141         no->key = temp->key; // Copiar o value do nó
142         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
143     }
144     return no;
145 }
146
147 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
148 void Remove(Tree * Tree, int value) {
149     Tree->root = RemoveNode(Tree->root, value);
150 }

```

no 38
right 64



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

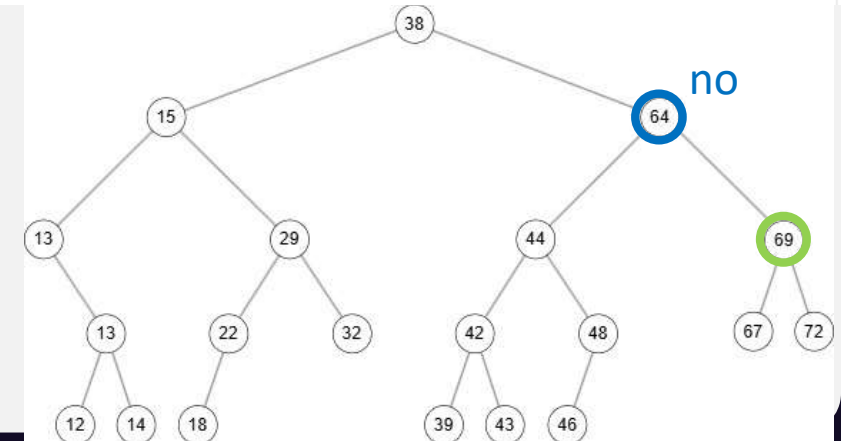
```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134
135         // Nó com dois filhos: obter o menor no da subárvore da Right
136         Node * temp = no->Right;
137         while (temp && temp->Left != NULL) {
138             temp = temp->Left;
139         }
140
141         no->key = temp->key; // Copiar o value do nó
142         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
143     }
144     return no;
145 }
146
147 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
148 void Remove(Tree * Tree, int value) {
149     Tree->root = RemoveNode(Tree->root, value);
150 }

```

no 38
right 64



```

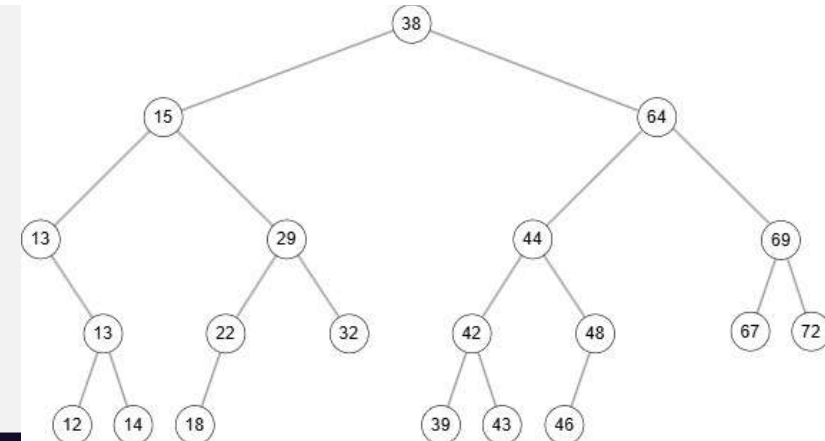
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

113 // Função para remover um elemento da árvore
114 Node * RemoveNode(Node * no, int value) {
115     if (no == NULL) {
116         return NULL; // Não encontrado
117     }
118     if (value < no->key) {
119         no->Left = RemoveNode(no->Left, value);
120     } else if (value > no->key) {
121         no->Right = RemoveNode(no->Right, value);
122     }
123     else {
124         // Nó encontrado
125         if (no->Left == NULL) {
126             Node * temp = no->Right;
127             free(no);
128             return temp;
129         } else if (no->Right == NULL) {
130             Node * temp = no->Left;
131             free(no);
132             return temp;
133         }
134
135         // Nó com dois filhos: obter o menor no da subárvore da Right
136         Node * temp = no->Right;
137         while (temp && temp->Left != NULL) {
138             temp = temp->Left;
139         }
140
141         no->key = temp->key; // Copiar o value do nó
142         no->Right = RemoveNode(no->Right, temp->key); // Remover o nó
143     }
144     return no;
145 }
146
147 // Função que remove nó e, quando tem 2 filhos, utiliza a metodologia do 'sucessor'
148 void Remove(Tree * Tree, int value) {
149     Tree->root = RemoveNode(Tree->root, value);
150 }

```




```

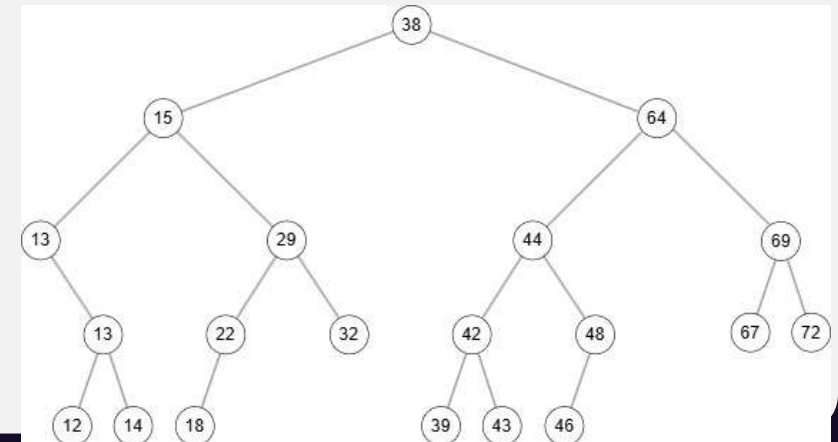
152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

```

Após remover 51:
38
|..15
|   |..11
|   |   |..13
|   |   |   |..12
|   |   |   |   |..14
|   |   |..29
|   |   |   |..22
|   |   |   |   |..18
|   |   |   |   |..32
|   |..64
|   |   |..44
|   |   |   |..42
|   |   |   |   |..39
|   |   |   |   |   |..43
|   |   |   |   |   |..48
|   |   |   |   |   |..46
|   |..69
|   |   |..67
|   |   |..72

```



```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```

Após remover 51:

```

38
|..15
|..11
|..13
|..12
|..14
|..29
|..22
|..18
|..32
|..64
|..44
|..42
|..39
|..43
|..48
|..46
|..69
|..67
|..72

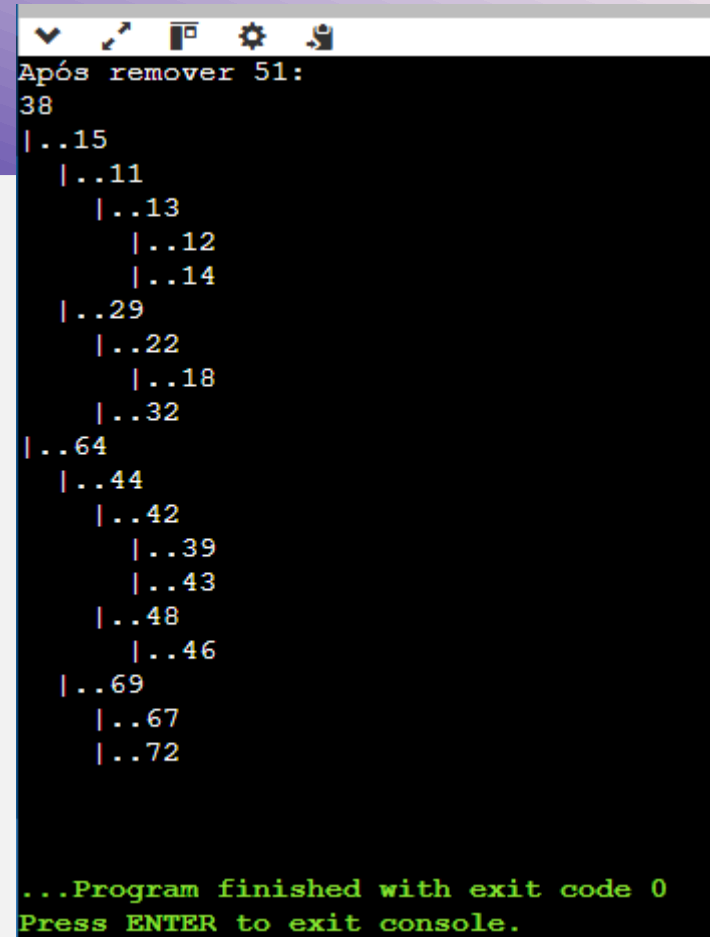
```

null


```

152 // Exemplo de uso
153 int main() {
154     Tree * minhaTree = CreateTree();
155     Insert(minhaTree, 38);
156     Insert(minhaTree, 51);
157     Insert(minhaTree, 44);
158     Insert(minhaTree, 15);
159     Insert(minhaTree, 64);
160     Insert(minhaTree, 42);
161     Insert(minhaTree, 11);
162     Insert(minhaTree, 69);
163     Insert(minhaTree, 48);
164     Insert(minhaTree, 13);
165     Insert(minhaTree, 29);
166     Insert(minhaTree, 46);
167     Insert(minhaTree, 39);
168     Insert(minhaTree, 22);
169     Insert(minhaTree, 32);
170     Insert(minhaTree, 67);
171     Insert(minhaTree, 43);
172     Insert(minhaTree, 12);
173     Insert(minhaTree, 18);
174     Insert(minhaTree, 72);
175     Insert(minhaTree, 14);
176
177     printf("Elementos da árvore:\n");
178     Print(minhaTree);
179
180     printf("A árvore está vazia? %s\n", IsEmpty(minhaTree) ? "Sim" : "Nao");
181     printf("Buscar 39: %s\n", Search(minhaTree, 39) ? "Encontrado" : "Nao encontrado");
182
183     Remove(minhaTree, 51);
184     printf("Após remover 51:\n");
185     Print(minhaTree);
186
187     Free(minhaTree);
188     return 0;
189 }

```



```

Após remover 51:
38
|..15
|..11
|..13
|..12
|..14
|..29
|..22
|..18
|..32
|..64
|..44
|..42
|..39
|..43
|..48
|..46
|..69
|..67
|..72

...Program finished with exit code 0
Press ENTER to exit console.

```